Vietnam National University, Ho Chi Minh City

University of Technology

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Mobile Application Development (CO3043)

Assignment 1: Specification Document

Entertainment App: Naomi

A Content-First, Ad-Free Webtoon & Manga Reader with Genre-Aware Instrumental Ambient Audio and Hybrid Offline-First Architecture

---

Instructor(s): Phan Tran Minh Khue

Class: CC01

Student: Duong Dang Khoa – 2352551

Ho Chi Minh City, November 2026

Contents

1 Problem Analysis & Requirements Elicitation **8**

1.1 Problem Statement 8

1.1.1 Domain Context 8

1.1.2 Core User Pain Points in Existing Platforms 8

1.2 Stakeholder Analysis 9

1.3 User Personas 10

1.3.1 Persona 1: Nguyen Tuan Kiet — "The Late-Night Immersionist" . . 10

1.3.2 Persona 2: Tran Phuong Thao — "The On-The-Go & Social Hunter" 11

1.3.3 Persona 3: Le Minh Duc — "The Fast-Paced Early Reader" 12

1.4 Competitor Analysis 12

1.5 Empirical Requirements Elicitation (Pilot Survey & Interviews) 13

1.5.1 Individual Respondent Matrix 14

1.5.2 Quantitative Statistical Synthesis & Architectural Implications 15

2 Functional Requirements Specification **17**

2.1 Methodology & MoSCoW Prioritisation 17

2.2 System Use Case Diagram 20

2.3 Detailed Use Case Specifications 20

2.3.1 UC-00: Authenticate & Manage User Account 22

2.3.2 UC-01: Discover & Search Comics 23

2.3.3 UC-02: View Comic Details & Access Chapters 24

2.3.4 UC-03: Read in Multiple Modes & Navigate 25

2.3.5 UC-04: Listen to Genre-Based Instrumental Music 26

2.3.6 UC-05: Preload Chapter Data & Save Reading Progress 27

2.3.7 UC-06: Download Chapter Offline & Handle Network Loss 28

2.3.8 UC-07: Manage Personal Library & UC-08: Customise App Settings 29

2.4 User Stories & Testable Acceptance Criteria (Gherkin BDD) 29

3 Non-Functional Requirements & Mobile Constraints **33**

3.1 SMART Non-Functional Design Philosophy 33

3.2 Master Non-Functional Requirements Matrix 33

3.3 Detailed Technical Justification across the Six Mobile Pillars 35
3.3.1 Pillar 1: Performance, Frame Budget & Preload Latency (NFR-01 to NFR-06) 35
3.3.2 Pillar 2: Offline Behaviour, Bounded Caching & Local Persistence (NFR-07 to NFR-09) 35
3.3.3 Pillar 3: Mobile Security & Cryptographic Privacy (NFR-10, NFR-11) 36
3.3.4 Pillar 4: System Permissions & Zero-Storage-Permission Visual Search (NFR-12, NFR-13) 36
3.3.5 Pillar 5 & Pillar 6: Platform Support, Battery & Network Efficiency (NFR-14 to NFR-18) 36
3.4 NFR Quality Assurance & Profiling Plan 37

4 System Architecture Design 38
4.1 High-Level Distributed Multi-Tier Architecture 38
4.2 In-App Clean Architecture & BLoC State Management 39
4.3 UML Sequence Diagrams for Core System Flows 40
4.3.1 Flow 1: Full-Screen Preloaded Reading with Genre Ambient Audio (UC-03, UC-04, UC-05) 40
4.3.2 Flow 2: Offline Chapter Download & Sandboxed Persistence (UC-06) 41
4.3.3 Flow 3: User Authentication & Hardware-Backed Auto-Login (UC-00) 41
4.4 Justified Technology Choices 41

5 Data & API Design 43
5.1 Hybrid Offline-First Data Architecture & Data Dictionary 43
5.2 Conceptual & Physical Entity-Relationship Diagrams 44
5.3 Detailed Relational Schema, 3NF/BCNF Normalisation Proof & RLS Security 46
5.3.1 Relational Schema Specifications (10 Tables) 46
5.3.2 Formal 3NF & Boyce-Codd Normal Form (BCNF) Proof 47
5.3.3 Database Security & Row-Level Security (RLS) Policies 47
5.4 On-Device 4-Tier Storage & Bidirectional Sync Strategy 48
5.5 RESTful API Contract Specification & Error Code Dictionary 49

6 UI/UX Design & High-Fidelity Mockups 53
6.1 Core Design Philosophy: “Less is More, Content-First” 53
6.2 Minimal Design Tokens & Accessibility Standards 53

6.3 Complete Screen Navigation & User Flow Architecture 54
6.4 High-Fidelity Mobile Mockups & Screen-by-Screen Specifications 55

7 Traceability Matrix & Document Quality Assurance 58
7.1 End-to-End Bidirectional Traceability Methodology 58
7.2 Functional Requirements Master Traceability Matrix 58
7.3 Non-Functional Requirements Traceability Matrix 60
7.4 Document Quality & Consistency Assurance Checklist 61

List of Figures

1 Sample empirical evidence extracted from the Google Forms user elicitation archive (`khao_sat_4_nguoi.pdf`). 15
2 UML 2.5 System Use Case Diagram for the Naomi Mobile Application (`UC-00` to `UC-08`). 20
3 C4 Container & Multi-Tier System Architecture Diagram of the Naomi Ecosystem. 39
4 Unidirectional data and state flow across Clean Architecture layers (`UI` ↔ `BLoC` ↔ `UseCase` ↔ `Repository`). 40
5 UML Sequence Diagram for Flow 1: Full-Chapter Preload (`S10`), 60 FPS Reading (`S05`), Genre Ambient Audio (`S06`), and Page Progress Sync. 40
6 UML Sequence Diagram for Flow 2: Sequential Chapter Page Download (`S11`) and Local SQLite Registration. 41
7 UML Sequence Diagram for Flow 3: Supabase JWT Authentication (`S01`) and Hardware-Backed Keystore Persistence. 41
8 Conceptual Entity-Relationship Diagram (Chen Notation) illustrating strong entities, the weak entity `PAGES` with partial key `page_number`, and binary relationships. 45
9 Physical Relational Database Schema (Crow's Foot Notation) enforcing 3NF/BCNF, Primary Keys (PK), Foreign Keys (FK), and Unique Constraints (UK). 45
10 Complete User Flow and Screen Navigation Graph across all 12 frames (`S01` to `S11`). 54

11 High-Fidelity Mockups (Part 1/3): Authentication (S01), 3-Block Home Feed (S02), Discover with AI Photo Search (S03), and Expanded Category Grid (S03b). 55

12 High-Fidelity Mockups (Part 2/3): Manga Detail (S04), Full-Chapter Preload State (S10), 100% Full-Screen Reader (S05), and Genre Instrumental Audio Sheet (S06). 56

13 High-Fidelity Mockups (Part 3/3): Two-Button Library (S07), Profile & Settings (S08), Offline Network Error State (S09), and Real-Time Download Progress State (S11). 56

List of Tables

1 Stakeholder classification, roles, and expectations for the Naomi ecosystem. . 10

2 Quantitative and qualitative competitor comparison across key mobile comic reader dimensions. 13

3 Raw empirical responses from the 4-participant requirements elicitation survey ($N = 4$). 14

4 Quantitative synthesis of elicitation findings and direct architectural implications. 16

5 Master Functional Requirements Matrix with MoSCoW prioritisation (FR-00 to FR-16). 17

6 Overview of the nine system Use Cases (UC-00 to UC-08). 21

7 Master Non-Functional Requirements Matrix (NFR-01 to NFR-18) with SMART thresholds and verification tools. 33

8 Quantitative QA verification and profiling protocol for Assignment 02 acceptance. 37

9 Comparative architectural trade-off analysis and justification of selected technologies. 42

10 System Data Dictionary across the 10 normalised tables of the Naomi database schema. 44

11 4-Tier On-Device Mobile Persistence and Eviction Strategy. 48

12 Catalogue of the 7 RESTful API Endpoints across MangaDex v5 and Supabase Backend. 50

13 System Error Code Dictionary and Deterministic UI/UX Recovery Actions. . 52

14 Minimalist Muted Soft Palette, Spatial Grid, and Accessibility Tokens. . . . 54

15 Comprehensive Specification of all 12 UI/UX Frames (S01 to S11) including Primary, Loading, Error, and Offline States. 57

16 Master Bidirectional Traceability Matrix for Functional Requirements (FR-00 to FR-16). 58

17 Bidirectional Traceability Matrix for Non-Functional Requirements (NFR-01 to NFR-18). 60

18 Cross-Section Consistency and Quality Assurance Verification Checklist. . . 61

1 Problem Analysis & Requirements Elicitation

1.1 Problem Statement

1.1.1 Domain Context

Digital comic consumption—spanning South Korean *Webtoons* (vertical long-strip format), Japanese *Manga* (paginated right-to-left or left-to-right format), and Chinese *Manhua*—has grown into one of the most dominant mobile entertainment habits among students and young professionals. Unlike desktop media consumption, mobile comic reading occurs across highly fragmented micro-sessions throughout the day: during lunch breaks between university lectures or office shifts, while commuting on public transit (buses, metro trains, ride-hailing vehicles), during short idle intervals, and during late-night wind-down routines before sleep (22:00–00:30).

1.1.2 Core User Pain Points in Existing Platforms

Despite massive demand, current mobile comic readers and unofficial aggregator websites suffer from seven severe technical and user-experience deficiencies identified through our domain investigation and user elicitation:

Pain Point 1: Intrusive and Harmful Advertisements: Free comic applications and pirated web aggregators aggressively inject auto-playing full-screen video advertisements with unmuted audio, persistent bottom/top banners that obscure artwork panels, and deceptive pop-up redirects to illicit gambling or malware sites. This shatters narrative immersion and wastes mobile cellular data.

Pain Point 2: Language Barriers and Unfinished Scanlations: Readers frequently encounter situations where an international series has released dozens of new chapters in its raw or English edition, whereas community Vietnamese translation (*scanlation*) groups have either fallen months behind or abandoned (*dropped*) the project entirely.

Pain Point 3: Fragmented Catalogs and Inability to Search by Image: Readers are forced to install three to four separate applications simultaneously to follow their favourite series. Furthermore, when users encounter a visually striking comic panel or screenshot on social media (*TikTok, Facebook, Threads*) without a title credit, existing mobile apps offer no visual reverse-search mechanism to identify the series.

Pain Point 4: Poor Offline Resilience and Restrictive Paywalls: Unstable 4G/5G cellular coverage during transit causes conventional online readers to freeze indefinitely on spinning loading indicators. Meanwhile, commercial apps either prohibit local chapter downloads or lock offline caching behind expensive VIP subscriptions and expiring 30-day rental coins, while unmanaged image caches silently bloat device storage.

Pain Point 5: Sub-optimal Scrolling Performance (Frame Drops): Poorly engineered readers decode and retain all high-resolution images of a 40–60 page Webtoon chapter in memory simultaneously, triggering severe UI jank, dropped frames (< 30 FPS), and Out-Of-Memory (OOM) crashes during rapid vertical scrolling.

Pain Point 6: Monotonous Single-Sense Reading Experience: Traditional digital comic reading is strictly static and silent, lacking subtle, genre-matched instrumental background audio (*ambient rain, soft piano, lo-fi beats, or orchestral tension*) that resonates with the emotional atmosphere of the narrative without forcing the user to manually juggle a separate music streaming app in the background.

Pain Point 7: Lack of Cross-Device Cloud Synchronisation: When users switch smartphones, read across a phone and tablet, or reinstall an application, they routinely lose their entire bookmarked library and exact page-level reading progress due to the absence of a secure cloud account synchronisation layer.

1.2 Stakeholder Analysis

Table 1 identifies the primary, internal, platform, external, and indirect stakeholders of the **Naomi** ecosystem along with their roles and technical expectations.

Table 1: Stakeholder classification, roles, and expectations for the Naomi ecosystem.

Stakeholder	Category	Role & Expectations for the Naomi System
End-Users (Mobile Comic Readers)	Primary	Expect a 100% ad-free reading environment, personal cloud account synchronisation, buttery-smooth 60 FPS vertical/horizontal rendering, automatic page-level progress tracking, genre-matched instrumental ambient audio, free 1-tap offline reading, and intelligent utilities (in-place AI translation, zero-permission visual search).
Development Team (Engineering)	Internal	Responsible for architecting the system using strict Clean Architecture and BLoC state management, enforcing bounded LRU disk caching ($\leq$ 500 MB) to prevent OOM crashes, integrating REST/BaaS APIs cleanly, and maintaining high unit-testability for Assignment 02.
Cloud Backend Provider (Supabase)	Platform	Provides enterprise-grade JWT authentication (`Supabase Auth`), cloud relational persistence (`PostgreSQL` with Row-Level Security), and object storage (`Supabase Storage`) for user profiles, bookmarks, reading history, and instrumental audio streams.
External Content API (MangaDex API v5)	External	Supplies open global comic metadata and decentralised `MangaDex@Home` CDN page delivery; requires the client application to strictly respect rate limits (5 req/s), exponential backoff, and caching headers.
Community Scanlation Groups	Indirect Partner	Expect their translated chapters and credits to be preserved intact and delivered within a strictly non-commercial, ad-free reader without unauthorized paywall monetisation.

1.3 User Personas

Grounded directly in our empirical survey responses and qualitative behavioural interviews (Section 1.5), three representative User Personas were constructed to guide feature prioritisation and UI/UX decisions.

1.3.1 Persona 1: Nguyen Tuan Kiet — "The Late-Night Immersionist"

Represents late-night readers (22:30–00:30, Survey Q1), highly receptive to genre-matched background audio (Survey Q3 & Q4), and deeply frustrated by loud video ads and scrolling lag (Survey Q2).

- **Demographics & Device:** 21-year-old third-year Information Technology undergraduate; uses a Xiaomi Redmi Note 12 (Android 13, 120 Hz AMOLED display).
- **Reading Context & Psychology:** Reads in bed every night between 22:30 and 00:30. For Kiet, late-night reading is not casual time-killing but an emotional decompression and healing ritual after intense coding sessions. He frequently opens Spotify in the back-

ground to play lo-fi or rain tracks while reading because gentle piano or rainfall deepens emotional scenes, whereas rhythmic instrumental tracks heighten action sequences.

- **Core Frustrations:**
  (a) *Immersion-shattering video ads:* Unmuted gambling video ads suddenly blasting at maximum volume at midnight, startling him awake and ruining his sleep.
  (b) *Scrolling stutter:* Frame drops when scrolling through 40–50 high-resolution Webtoon strips.
  (c) *Battery & thermal drain:* Running Spotify alongside a heavy comic app causes his phone to overheat and drain battery rapidly.
- **Goals with Naomi:** A 100% ad-free reader with an eye-soothing AMOLED Dark Mode (`#111116`), built-in instrumental ambient audio automatically matched to the comic's genre with a minimal volume slider, and seamless toggling between 60 FPS vertical Webtoon scrolling and horizontal Manga page-flipping.

1.3.2 Persona 2: Tran Phuong Thao — "The On-The-Go & Social Hunter"

Represents opportunistic daytime readers (50% in Survey Q1), frustrated by cellular dropouts and paywalled offline modes (Survey Q2), and eager for visual image search (Survey Q5).

- **Demographics & Device:** 20-year-old second-year Multimedia Communications student in Ho Chi Minh City; uses a Samsung Galaxy A54 (Android 14, capped monthly 4G data plan).
- **Reading Context & Psychology:** Reads during 10–15 minute breaks between classes or while commuting on buses and ride-hailing bikes. Actively browses TikTok, Facebook, and Threads, where she frequently saves screenshots of aesthetically striking comic panels that lack title captions.
- **Core Frustrations:**
  (a) *Dead-ends when searching by image:* Having to post uncredited screenshots in community groups and wait hours for someone to identify the comic title.
  (b) *Cellular network freezes:* Online apps freezing on an endless loading spinner whenever the bus passes through weak 4G reception zones.
  (c) *Paywalled offline downloads:* Commercial apps locking offline chapter downloads behind paid VIP tiers or expiring coins.
- **Goals with Naomi:** Zero-permission Visual Search via the OS Photo Picker to identify comics from saved screenshots instantly, free 1-tap offline chapter downloading over home Wi-Fi for lag-free transit reading, and a clean two-category Personal Library (*Favorites* and *Offline Reading*).

1.3.3 Persona 3: Le Minh Duc — "The Fast-Paced Early Reader"

*Represents lunch-break readers (Survey Q1) who follow multiple ongoing series, encounter untranslated latest chapters (Survey Q2), and demand in-place AI translation (*100% *in Survey Q5) and exact page-level history (*75% *in Survey Q5).*

- **Demographics & Device:** 23-year-old junior software engineer working in Hanoi; uses a Google Pixel 7 (Android 14).
- **Reading Context & Psychology:** Reads during his office lunch break (11:45–12:45) or after overtime. Follows 5–7 ongoing action/fantasy series simultaneously and wants to read newly released chapters immediately upon international publication.
- **Core Frustrations:**
  (a) *Abandoned translations:* When an international series reaches Chapter 150 but local scanlation groups stop at Chapter 100, switching back and forth between the comic app and Google Translate destroys reading flow.
  (b) *Lost reading position:* Abruptly closing the app when work resumes and returning the next day only to find the app forgot which page he was on within a 50-page chapter.
- **Goals with Naomi:** Automatic page-level reading history synced to a prominent *Continue Reading* hero card on the Home screen, in-place AI chapter translation (`FR-12`), and genre-based recommendations upon finishing a chapter.

1.4 Competitor Analysis

To position **Naomi** objectively within the mobile entertainment market, we conducted a comparative evaluation against the two dominant market leaders representing distinct segments: **LINE Webtoon** (the leading global official platform) and **MangaToon** (a widely used commercial aggregator in Vietnam). Table 2 synthesises the technical and experiential comparison.

Table 2: Quantitative and qualitative competitor comparison across key mobile comic reader dimensions.

Evaluation Dimension	LINE Webtoon (Global)	MangaToon (Vietnam)	Naomi (Proposed System)
Advertisement Intrusiveness	Moderate (mandatory video ads for Daily Pass, promotional pop-ups)	**Severe** (pop-up banners, gambling redirects, panel-obscuring ads)	**100% Ad-Free** (zero ad SDKs, zero pop-ups or paywalls)
Interactive Ambient Audio	Absent (limited to rare paid promotional episodes)	Completely absent	**Built-in genre-matched instrumental ambient audio player**
Offline Reading & Storage	Restricted (requires coins/locks, auto-expires after 30 days)	Restricted (locked behind paid VIP membership tiers)	**Free 1-tap offline download** persisted in local SQLite & sandboxed storage
Page-Level Progress Sync	Supported	Supported	**Real-time page-level tracking** ($\leq$ 50 ms local SQLite + Supabase LWW cloud sync)
Reading Mode Flexibility	Vertical scroll only (Webtoon)	Primarily vertical scroll	**Dual-Mode Toggle:** Vertical Webtoon scroll $\leftrightarrow$ Horizontal Manga pagination
Content Ecosystem	Proprietary closed catalog	Proprietary closed catalog	**Open MangaDex REST API v5** + custom Supabase backend
Smart Utilities (AI / Visual Search)	Not supported	Not supported	**Zero-permission Visual Search + In-place AI Translation** (Could-have roadmap)
UI/UX & Rendering	Heavyweight multi-tab chrome	Cluttered with coin shops, daily quests, and promotional tabs	**Minimalist Content-First UI**, muted palette, full-chapter pre-buffering at 60 FPS

1.5 Empirical Requirements Elicitation (Pilot Survey & Interviews)

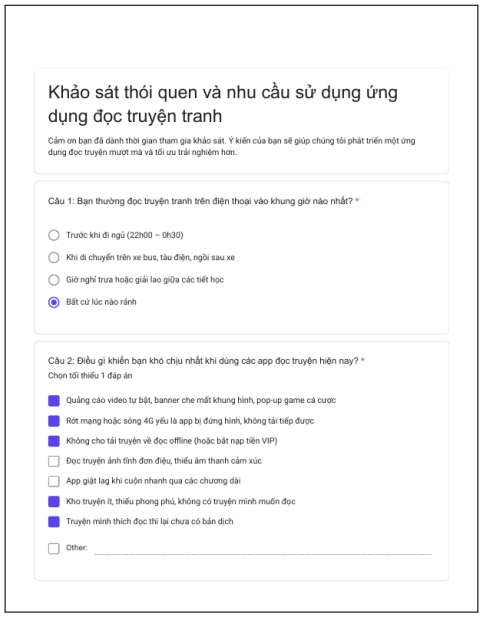
Following the requirements engineering methodology of Sommerville (Chapter 4 – *Requirements Elicitation and Analysis*), we conducted a mixed-methods empirical study combining a structured **Pilot Survey** via Google Forms and **In-Depth Qualitative Elicitation** across $N = 4$ real target users spanning all three reader segments (full original response evidence is archived in `document/khao_sat_4_nguoi.pdf` and summarised visually in Figure 1).

1.5.1 Individual Respondent Matrix

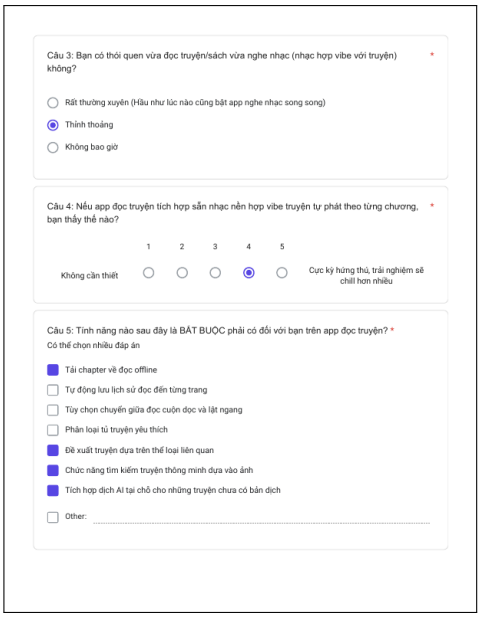
Table 3 presents the exact raw responses recorded across the four participants (R1–R4) for all six survey instruments.

Table 3: Raw empirical responses from the 4-participant requirements elicitation survey ($N = 4$).

Survey Question	Respondent 1 (R1)	Respondent 2 (R2)	Respondent 3 (R3)	Respondent 4 (R4)
Q1: Primary reading time window	Whenever free during the day	Whenever free during the day	Lunch break / between classes	Late night before sleep (22:00–00:30)
Q2: Most frustrating issues in current apps *(Multi-select)*	• Intrusive/gambling ads • Network drop freezes • No offline download • Small catalog • Missing translations	• Intrusive/gambling ads • Network drop freezes • No offline download • Scrolling lag/jank • Small catalog • Missing translations	• Intrusive/gambling ads	• Intrusive/gambling ads • Scrolling lag/jank • Small catalog • Missing translations
Q3: Habit of listening to music while reading	Occasionally	Very frequently (runs Spotify in parallel)	Occasionally	Occasionally
Q4: Interest in genre-matched ambient audio (1–5 Likert)	**4 / 5** (High interest)	**4 / 5** (High interest)	**3 / 5** (Moderate)	**4 / 5** (High interest)
Q5: Must-have features on a new comic app *(Multi-select)*	• Offline chapter download • Genre recommendations • Visual image search • In-place AI translation	• Page-level history sync • Library categorisation • Visual image search • In-place AI translation	• Page-level history sync • Genre recommendations • In-place AI translation	• Page-level history sync • Vertical/Horizontal toggle • Genre recommendations • In-place AI translation
Q6: Willingness to switch to Naomi	**Definitely Yes**	**Definitely Yes**	**Definitely Yes**	**Definitely Yes**



(a) Survey Form – Questions 1 & 2 (Respondent 1)



(b) Survey Form – Questions 3, 4 & 5 (Respondent 1)

Figure 1: Sample empirical evidence extracted from the Google Forms user elicitation archive (`khao_sat_4_nguoi.pdf`).

1.5.2 Quantitative Statistical Synthesis & Architectural Implications

Table 4 consolidates the quantitative distribution of user pain points and feature demands alongside the concrete engineering decisions derived for **Naomi**.

Table 4: Quantitative synthesis of elicitation findings and direct architectural implications.

Empirical Finding / Metric	Prevalence	Direct Engineering & Specification Decision
Intrusive video/banner/gambling ads (Q2)	**100.0%** (4/4)	Strict **100% Ad-Free commitment**; zero advertising or tracking SDKs; monetisation/IAP explicitly excluded as `WON'T-HAVE` (`FR-15`).
Habit of pairing music with reading (Q3) & **Ambient Audio Rating** (Q4)	**100.0%** (4/4) Mean: **3.75 / 5.0**	Core differentiator (**Unique Value Proposition**): integrate `just_audio` streaming from Supabase Storage matched to comic genres (`FR-05`, `FR-09`, `UC-04`).
Missing translations (Q2) & **In-place AI Translation demand** (Q5)	**75%–100%** (3–4/4)	Specify in-place AI chapter translation overlay (`FR-12`) on the Reader top toolbar (`S05` Icon $\star$) as a high-value `COULD-HAVE` capability.
Exact page-level reading history (Q5)	**75.0%** (3/4)	Persist `last_page_read` and `read_percentage` asynchronously in SQLite ($\leq$ 50 ms) with bidirectional Supabase cloud sync (`FR-04`, `NFR-09`) and surface on the `S02` Home Hero card.
Limited catalog (Q2) & **Genre-based recommendations** (Q5)	**75.0%** (3/4)	Integrate the open **MangaDex REST API v5** catalog (`FR-01`, `FR-08`) with horizontal *Trending* and *Genre* discovery carousels (`S03`, `S03b`).
Cellular network freezes & lack of free offline mode (Q2, Q5)	**50.0%** (2/4)	Design a **Hybrid Offline-First Architecture** with full-chapter pre-buffering (`S10`), 1-tap local chapter downloads (`FR-06`, `S11`), and $\leq$ 300 ms offline fallback (`FR-13`, `S09` $\rightarrow$ `S07`).
Scrolling lag on long chapters (Q2)	**50.0%** (2/4)	Enforce **60 FPS rendering** ($\leq$ 16.6 ms/frame, `NFR-03`) via Flutter Impeller/Skia virtualised viewport lists and strict file-system media storage (never storing image BLOBs in SQLite).
Visual image search from screenshots (Q5)	**50.0%** (2/4)	Provide camera icon trigger on `S03` invoking the OS **System Photo Picker** (`FR-11`), achieving visual search with **zero storage permissions** (`NFR-12`).
Willingness to adopt Naomi over existing apps (Q6)	**100.0%** (4/4)	Validates strong product-market fit provided the core Must-Have and Should-Have baseline is delivered cleanly in Assignment 02.

2 Functional Requirements Specification

2.1 Methodology & MoSCoW Prioritisation

To rigorously control project scope, guarantee 100% implementation feasibility in Assignment 02, and maintain direct alignment with the empirical user elicitation findings from Section 1, all functional requirements of the **Naomi** platform are classified using the **MoSCoW Prioritisation Framework**:

- **MUST-HAVE (Core Acceptance Baseline):** Non-negotiable functional capabilities required for the system to operate end-to-end; 100% of Must-Have requirements must pass acceptance verification in Assignment 02.
- **SHOULD-HAVE (High-Value Experience Enhancers):** Important features that elevate user experience, resilience, and personalisation to a production-grade standard (90–100% target rubric band).
- **COULD-HAVE (Innovative Smart Extensions):** Advanced AI/computer-vision capabilities (Visual Search and In-Place AI Chapter Translation) implemented as modular prototypes without compromising core stability.
- **WON'T-HAVE (Explicitly Excluded Scope):** Features intentionally omitted from the project boundary to preserve our non-commercial, ad-free, distraction-free *Content-First* philosophy.

Table 5 defines the complete catalogue of 17 Functional Requirements (`FR-00` through `FR-16`).

Table 5: Master Functional Requirements Matrix with MoSCoW prioritisation (`FR-00` to `FR-16`).

FR ID	Requirement Name	MoSCoW	System Business Capability Description
FR-00	**User Account Authentication**	**MUST-HAVE**	Enables users to register, sign in, and request password recovery via Email/-Password (`Supabase Auth`). Persists encrypted JWT sessions in hardware-backed storage. Excludes third-party OAuth social logins to maintain provider independence.

Continued on next page

Table 5 – continued from previous page

FR ID	Requirement Name	MoSCoW	System Business Capability Description
FR-01	**Comic Discovery & Search (MangaDex API)**	**MUST-HAVE**	Searches comics by title or author keyword; organises horizontal discovery rows by *Trending* and *Genre* (*Fantasy, Action*, etc.); supports expanding any row into a full paginated grid (`S03b`).
FR-02	**Full-Screen Distraction-Free Reader**	**MUST-HAVE**	Renders chapter pages across 100% of the viewport; provides a single translucent top icon toolbar for chapter navigation and a 1-tap *Focus Mode* that hides all UI chrome.
FR-03	**Flexible Dual Reading Mode Toggle**	**MUST-HAVE**	Allows instant in-reader switching between continuous vertical scrolling (*Webtoon long-strip mode*) and horizontal paginated flipping (*traditional Manga mode*).
FR-04	**Auto-Save Progress & Instant Resume**	**MUST-HAVE**	Automatically records the exact chapter and page number (`last_page_read`) into local SQLite and synchronises to Supabase Cloud; surfaces a 1-tap *Continue* hero card on the Home screen (`S02`).
FR-05	**Genre-Matched Instrumental Ambient Audio**	**MUST-HAVE**	Automatically recommends and streams instrumental background audio tracks matching the primary genre of the active comic to foster an immersive, relaxing reading atmosphere.
FR-06	**Offline Chapter Download**	**MUST-HAVE**	Downloads all page image assets of selected chapters to sandboxed device storage for offline reading; displays real-time percentage and byte progress (`S11`).
FR-07	**Personal Library Management**	**MUST-HAVE**	Provides a dedicated two-button Library hub (`S07`) separating *Favorites* (cloud-synced bookmarked series) and *Offline Reading* (locally saved chapters with storage footprint metrics).
FR-08	**Consolidated Home Feed & Recommendations**	**SHOULD-HAVE**	Presents a streamlined three-block Home screen (`S02`): (1) *Continue Reading* hero card, (2) *Recent* horizontal carousel, and (3) *Recommended* horizontal carousel based on genres.

Continued on next page

Table 5 – continued from previous page

FR ID	Requirement Name	MoSCoW	System Business Capability Description
FR-09	**Minimalist Audio Sheet Controls**	**SHOULD-HAVE**	Provides a clean bottom sheet (S06) with Play/Pause toggle, continuous volume slider, and track selection among genre-matched instrumentals; auto-pauses on phone calls or app backgrounding.
FR-10	**Comic Detail View & Quick Chapter Actions**	**SHOULD-HAVE**	Displays complete comic metadata (cover, author, rating, genres, synopsis), four primary action buttons (*Read First, Read Latest, Favorite, Download*), and the full chapter list (S04).
FR-11	**Smart Visual Search via System Photo Picker**	**COULD-HAVE**	Allows users to select a comic panel screenshot via the OS `System Photo Picker` (requiring zero storage permissions) to identify the matching series (S03).
FR-12	**In-Place AI Chapter Translation**	**COULD-HAVE**	Integrates an on-demand AI translation action (S05 Icon ⋆) that translates dialogue text across the currently open chapter into the user's target language.
FR-13	**Offline Network Loss Fallback**	**SHOULD-HAVE**	Detects loss of Internet connectivity when opening an uncached chapter, displays a friendly offline error state (S09), and provides a direct CTA button routing to the Offline Library (S07).
FR-14	**User Profile & System Settings Customisation**	**SHOULD-HAVE**	Manages user profile details, appearance theme (*System / Light / Dark*), UI language (*English / Vietnamese*), disk cache inspection/clearing, notification toggles, and sign-out (S08).
FR-15	**In-App Purchases (IAP) & Virtual Coins**	**WON'T-HAVE**	Explicitly excluded to uphold a 100% non-commercial, paywall-free reading environment.
FR-16	**In-App Social Feed & 1-on-1 Direct Messaging**	**WON'T-HAVE**	Explicitly excluded to eliminate social clutter and keep architectural complexity focused on reading performance.

2.2 System Use Case Diagram

Figure 2 illustrates the UML 2.5 Use Case Diagram modelling all interactions between the primary human actor (**Reader**), the nine core use cases (`UC-00` to `UC-08`) inside the **Naomi Mobile Application** boundary, and the two external system actors (**Supabase Backend** and **MangaDex REST API**). Mandatory sub-flows are linked via `«include»` dependencies (`UC-03` includes `UC-05`), while optional contextual enrichments are modelled via `«extend»` dependencies (`UC-04` extends `UC-03`; `UC-02` extends `UC-01`; `UC-06` extends `UC-07`).

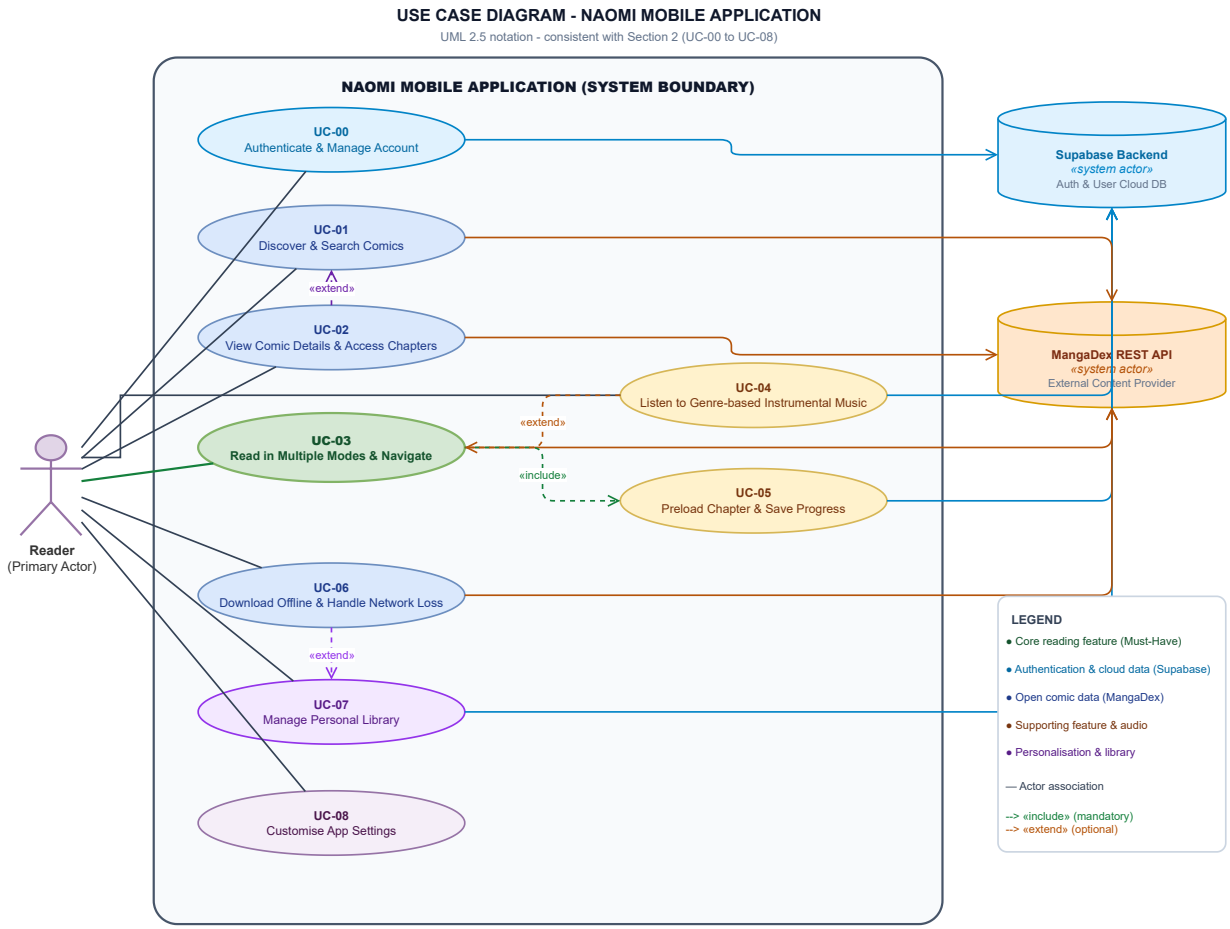


Figure 2: UML 2.5 System Use Case Diagram for the Naomi Mobile Application (`UC-00` to `UC-08`).

2.3 Detailed Use Case Specifications

Table 6 summarises the scope of the nine system use cases, followed by their complete structured specifications (Actors, Preconditions, Postconditions, Main Success Scenario, Alternative Flows, and Exception Flows).

Table 6: Overview of the nine system Use Cases (UC-00 to UC-08).

UC ID	Use Case Name	Primary Actor	Business Goal
UC-00	Authenticate & Manage Account	Reader	Sign up, sign in, or reset password via Email/Password; persist encrypted JWT session via Supabase Auth.
UC-01	Discover & Search Comics	Reader	Search comics by keyword, browse *Trending* and *Genre* rows, expand full lists, or search by image (FR-11).
UC-02	View Comic Details & Access Chapters	Reader	Inspect comic metadata, trigger *Read First / Read Latest*, bookmark to *Favorites*, or initiate chapter downloads.
UC-03	Read in Multiple Modes & Navigate	Reader	Read full-screen pages, toggle Vertical Webtoon ↔ Horizontal Manga, switch chapters, and activate *Focus Mode*.
UC-04	Listen to Genre-Based Instrumental Music	Reader	Stream instrumental ambient tracks matched to the comic's genre; control playback, volume, and track selection.
UC-05	Preload Chapter & Save Progress	System / Reader	Pre-buffer 100% of chapter pages before opening the reader (S10); continuously persist page progress to SQLite & Cloud.
UC-06	Download Offline & Handle Network Loss	Reader	Download chapter page assets into sandboxed local storage (S11); route gracefully to Offline Library on network loss (S09).
UC-07	Manage Personal Library	Reader	Access and manage the two-button Personal Library (S07): cloud-synced *Favorites* and local *Offline Reading*.
UC-08	Customise App Settings	Reader	Configure appearance theme (*System/Light/Dark*), UI language, disk cache cleanup, notifications, and sign-out (S08).

2.3.1 UC-00: Authenticate & Manage User Account

Field	Specification Details
Use Case ID & Name	**UC-00: Authenticate & Manage User Account** (Maps to `FR-00`, `NFR-05`, `NFR-10`)
Primary / Secondary Actors	**Primary:** Reader | **Secondary:** Supabase Auth Server (`POST /auth/v1/token`)
Preconditions	Device has active Internet connectivity; user is unauthenticated on screen `S01` (*Sign In*).
Postconditions	Authenticated session is established; JWT `access_token` and `refresh_token` are encrypted via AES-256 in `FlutterSecureStorage` (Android Keystore / iOS Keychain).
Main Success Flow (Sign In)	1. User opens the application and lands on the *Sign In* screen (`S01`). 2. User enters a valid email address and password and taps **Sign In**. 3. System validates input syntax client-side (RFC 5322 email format, non-empty password). 4. System dispatches an HTTPS TLS 1.3 authentication request to Supabase Auth (`API 04`). 5. Upon receiving `HTTP 200 OK` ($\leq$ 1.0 s), system stores the JWT pair in `FlutterSecureStorage`, loads the user profile, and navigates to the *Home* screen (`S02`).
Alternative Flows	• **00a – Account Registration (*Sign Up*):** User taps *Sign Up* on `S01` $\rightarrow$ submits valid email and new password $\rightarrow$ system calls `POST /auth/v1/signup`, creates the `users` profile row via database trigger/upsert, and signs the user in automatically. • **00b – Password Recovery (*Forgot Password?*):** User taps *Forgot Password?* $\rightarrow$ enters registered email $\rightarrow$ Supabase dispatches a password reset link via email. • **00c – Auto-Login on Warm/Cold Launch:** If a valid token exists in `FlutterSecureStorage`, `AuthBloc` restores the session silently and opens `S02` directly ($\leq$ 0.8 s warm start).
Exception Flows	• **00e – Invalid Credentials (`HTTP 400`):** System displays an inline validation message (*"Invalid email or password"*) while preserving the typed email address for correction.

2.3.2 UC-01: Discover & Search Comics

Field	Specification Details
Use Case ID & Name	**UC-01: Discover & Search Comics** (Maps to `FR-01`, `FR-11`, `NFR-12`)
Primary / Secondary Actors	**Primary:** Reader | **Secondary:** MangaDex REST API v5 (`GET /manga`)
Preconditions	User navigates to the *Discover* tab (`S03`).
Postconditions	Matching comics or expanded category grids (`S03b`) are rendered with real cover artwork.
Main Success Flow	1. User selects the *Discover* tab (`S03`) from the bottom navigation bar. 2. System queries `GET /manga` (`API 01`) to populate horizontal scroll carousels for **Trending** and specific **Genres** (*Fantasy, Action*, etc.). 3. User either types a query into the search bar (*"Search titles, authors..."*) or taps the expand chevron (`>`) next to any section header. 4. System transitions to the *Expanded List* screen (`S03b`), displaying a 2-column vertical grid of matching titles with infinite pagination.
Alternative Flows	• **01a – Zero-Permission Visual Search (`FR-11`):** User taps the Camera icon inside the `S03` search bar → system invokes the OS `System Photo Picker` (requiring zero storage permissions) → user selects a comic panel screenshot → system extracts visual features/title match and displays the corresponding comic result.
Exception Flows	• **01e – Empty Result or Rate Limit (`HTTP 429`):** If no titles match, system renders a clean Empty State; if MangaDex returns `HTTP 429`, the `Dio` interceptor applies exponential backoff automatically.

2.3.3 UC-02: View Comic Details & Access Chapters

Field	Specification Details
Use Case ID & Name	**UC-02: View Comic Details & Access Chapters** (Maps to `FR-10`, `FR-06`, `FR-07`)
Primary / Secondary Actors	**Primary:** Reader | **Secondary:** MangaDex API (`GET /manga/{id}/feed`) & Supabase
Preconditions	User taps a comic card on *Home* (`S02`), *Discover* (`S03`/`S03b`), or *Library* (`S07`).
Postconditions	Comic metadata and chapter list are displayed on `S04`; bookmark or download actions are persisted.
Main Success Flow	1. User taps a comic card (e.g., *Frieren: Beyond Journey's End*). 2. System opens *Manga Detail* (`S04`) and fetches metadata plus the chapter feed (`API 02`). 3. System renders cover art, title, author/artist, star rating ($\star$ 4.9), genre tags, synopsis, the 4-button action bar (**Read First**, **Read Latest**, ♡ **Favorite**, ↓ **Download**), and the chapter list showing saved/read badges. 4. User taps **Read First**, **Read Latest**, or any specific chapter row to begin reading (invoking `UC-05` → `UC-03`).
Alternative Flows	• **02a – Toggle Favorite Bookmark:** User taps ♡ **Favorite** → button highlights in soft coral red (`#E05353`), upserting into local SQLite `bookmarks` and syncing to Supabase (`API 06`).

2.3.4 UC-03: Read in Multiple Modes & Navigate

Field	Specification Details
Use Case ID & Name	**UC-03: Read in Multiple Modes & Navigate** (Maps to FR-02, FR-03, FR-12, NFR-03)
Primary / Secondary Actors	**Primary:** Reader | **Included UC:** UC-05 (Preload & Save Progress)
Preconditions	Chapter pages have been pre-buffered via S10 (UC-05) or loaded from local offline storage.
Postconditions	User reads at $\geq$ 60 FPS; reading mode preference and current page index are saved continuously.
Main Success Flow	1. System opens the *Full Screen Reader* (S05) with artwork occupying 100% of the viewport over an AMOLED dark background (#111116). 2. A single translucent top toolbar (rgba(22,22,28,0.90)) provides icon-only controls: • **Back (<):** Returns to S04 or S02. • **Chapter Switcher (< Ch. 24 ▼ >):** Steps to previous/next chapter or opens a dropdown to jump directly to any chapter. • **Music Icon (♫):** Opens the *Audio Sheet* (S06, invoking UC-04). • **Focus Mode Icon (⊙):** Hides the top toolbar completely for 100% unobstructed artwork viewing; tapping the canvas restores the toolbar. • **Language Icon (⊕):** Selects target translation/UI language. • **AI Translate Icon ($\star$ – FR-12):** Translates dialogue in the current chapter into the selected language in-place. • **Reader Mode Toggle Icon ($\leftrightarrows$ – FR-03):** Switches instantly between continuous Vertical Webtoon Scroll and Horizontal Manga Page Flip.
Exception Flows	• **03e – Mid-Chapter Network Drop:** Because 100% of the active chapter was pre-buffered in UC-05, the user continues reading the entire chapter without interruption.

2.3.5 UC-04: Listen to Genre-Based Instrumental Music

Field	Specification Details
Use Case ID & Name	**UC-04: Listen to Genre-Based Instrumental Music** (Maps to `FR-05`, `FR-09`, `NFR-16`, `NFR-17`)
Primary / Secondary Actors	**Primary:** Reader | **Secondary:** Supabase Storage & DB (`GET /rest/v1/ambient_tracks`)
Preconditions	User is actively reading a chapter in `S05` and taps the Music icon (🎵).
Postconditions	Selected instrumental track loops smoothly at the chosen volume or pauses cleanly.
Main Success Flow	1. User taps the Music icon (🎵) on the `S05` top toolbar. 2. System slides up the minimalist *Audio Sheet* (`S06`) over the dimmed reader canvas. 3. System displays the active instrumental track matched to the comic's genre (e.g., *"Winds of Aureole (Instrumental)"* for *Fantasy*), a 44 px circular Play/Pause button, a continuous volume slider (defaulting to a gentle 30%), and a 3-track list of related genre instrumentals. 4. User adjusts volume, pauses/resumes, or taps another track in the list to switch audio immediately while continuing to read.
Exception Flows	• **04e – Audio Focus Loss (Phone Call / App Backgrounded):** `AudioSession` detects OS focus loss and pauses playback within $\leq$ 100 ms (`NFR-17`) to prevent audio conflict and battery drain.

2.3.6 UC-05: Preload Chapter Data & Save Reading Progress

Field	Specification Details
Use Case ID & Name	**UC-05: Preload Chapter Data & Save Reading Progress** (Maps to `FR-02`, `FR-04`, `NFR-04`, `NFR-09`)
Primary / Secondary Actors	**Primary:** System / Reader | **Secondary:** MangaDex@Home CDN (`API 03`) & Supabase (`API 05`)
Preconditions	User triggers chapter opening from `S02` (*Continue*) or `S04` (*Manga Detail*).
Postconditions	All chapter pages are pre-buffered in cache; `reading_history` is updated locally and queued for cloud sync.
Main Success Flow	1. System displays the *Full Chapter Preload* screen (`S10`) over an AMOLED dark backdrop with a pulsing cover thumbnail and progress bar (*"Loading full chapter... 42 / 42 pages ready (100%)"*). 2. If the chapter is already saved offline in SQLite, `S10` completes instantaneously; otherwise, system resolves `GET /at-home/server/{chapterId}` ($\leq$ 1.5 s initial latency) and pre-buffers all chapter pages concurrently. 3. Upon reaching 100%, system transitions automatically to `S05` at the user's last read page (e.g., Page 18/42). 4. As the user scrolls or flips pages, `ReaderBloc` writes `last_page_read` and `read_percentage` asynchronously to local SQLite ($\leq$ 50 ms) and upserts to Supabase (`API 05`).

2.3.7 UC-06: Download Chapter Offline & Handle Network Loss

Field	Specification Details
Use Case ID & Name	**UC-06: Download Chapter Offline & Handle Network Loss** (Maps to `FR-06`, `FR-13`, `NFR-08`)
Primary / Secondary Actors	**Primary:** Reader | **Secondary:** MangaDex@Home CDN & Local Sandboxed Storage
Preconditions	User views a comic on `S04` or attempts to open an uncached chapter while offline.
Postconditions	Chapter images are persisted in `ApplicationDocumentsDirectory` and registered in `offline_downloads`; or user is routed cleanly to `S07` during network loss.
Main Success Flow (Download)	1. On `S04`, user taps ↓ **Download** or the download arrow on a specific chapter row (e.g., Ch. 24). 2. System transitions to `S11` (*Download in Progress*), streaming image bytes sequentially to sandboxed storage while displaying a green progress bar (*"Downloading... 24.5 MB / 36 MB – 68%"*) and bottom toast notification. 3. Upon completion, system updates SQLite `offline_downloads.download_status = 'completed'` and marks the chapter badge as **Saved**.
Exception Flow (Offline Fallback)	• **06e – Network Disconnected on Uncached Chapter (`S09`):** If the device has no Internet connection and the requested chapter is not in `offline_downloads`, system detects offline status within $\leq$ 300 ms (`NFR-08`), displays `S09` (*"Oops! No internet connection"*), and provides the primary CTA button **Go to Offline Library**, routing directly to `S07`.

2.3.8 UC-07: Manage Personal Library & UC-08: Customise App Settings

Field	Specification Details
UC-07: Manage Personal Library	• **Goal & Mapping:** Centralised personal collection hub (`FR-07`, Screen `S07`). • **Main Flow:** User taps the *Library* tab (`S07`). System renders exactly two large 20 px-radius action cards: 1. **Favorites (♡ – *"12 stories marked"*):** Opens the cloud-synced list of bookmarked series from Supabase/SQLite. 2. **Offline Reading (↓ – *"8 chapters saved · 280 MB"*):** Opens locally downloaded chapters available for zero-network reading and allows single-chapter deletion to free disk space.
UC-08: Customise App Settings	• **Goal & Mapping:** Personalise system preferences and manage storage/session (`FR-14`, `NFR-07`, Screen `S08`). • **Main Flow:** User taps the *Profile* tab (`S08`), displaying user identity (*Kiet Nguyen – reader.kiet@gmail.com*) and five configuration controls: (1) **Appearance** segmented control (*System / Light / Dark*), (2) **Language** selector (*English / Tieng Viet*), (3) **Storage & Cache** indicator (*142 MB cached*) with 1-tap *Clear Cache* button, (4) **Notifications** toggle switch, and (5) **Sign Out** action (wiping JWT tokens from `FlutterSecureStorage` and returning to `S01`).

2.4 User Stories & Testable Acceptance Criteria (Gherkin BDD)

To establish an unambiguous, directly verifiable baseline for Assignment 02 Unit and Acceptance Testing (Rubric Criterion 1.2 & 2.5), every Use Case is formalised into a User Story accompanied by **Behaviour-Driven Development (BDD)** scenarios in standard `Given-When-Then` Gherkin syntax.

Listing 1: US-00 & US-01: Authentication and Comic Discovery Acceptance Criteria.

```
Feature: US-00 User Account Authentication (Maps to FR-00, UC-00, S01)
  As a mobile reader, I want to sign in, sign up, or recover my password
     via Email
  So that my bookmarks and page-level reading history sync securely
     across devices.

  Scenario: Successful sign-in with valid credentials
    Given the user is on the "S01 Sign In" screen
    When the user enters email "reader.kiet@gmail.com" and a valid
       password
    And taps the "Sign In" button
    Then the system authenticates via Supabase Auth within 1.0 second
    And encrypts the JWT access and refresh tokens inside
       FlutterSecureStorage
```

```
    And navigates the user to the "S02 Home" screen

  Scenario: New account registration and automatic session start
    Given the user is on the "S01 Sign In" screen and taps "Sign Up"
    When the user submits a valid new email and password
    Then the system creates a new account on Supabase Auth and inserts a
       "users" record
    And automatically establishes an authenticated session on "S02 Home"

Feature: US-01 Comic Discovery, Expanded Lists & Visual Search (Maps to
   FR-01, FR-11, UC-01, S03, S03b)
  As an exploratory reader, I want to search comics by text or screenshot
     and browse Trending/Genres
  So that I can quickly find compelling series to read.

  Scenario: Browsing Trending and Genre carousels and expanding full lists
    Given the user is on the "S03 Discover" screen
    When the user views the discovery sections
    Then horizontal carousels for "Trending" and genres ("Fantasy",
       "Action") are displayed
    When the user taps the expand chevron ">" next to "Trending"
    Then the system opens "S03b Expanded List" displaying a full 2-column
       paginated grid

  Scenario: Zero-permission Visual Search via System Photo Picker
    Given the user taps the Camera icon inside the "S03 Discover" search
       bar
    When the OS System Photo Picker opens without requesting storage
       permissions
    And the user selects a saved manga panel screenshot
    Then the system queries matching visual metadata and displays the
       corresponding comic series
```

Listing 2: US-02, US-03, US-04 & US-05: Detail View, Full-Screen Reader, Ambient Audio, and Preload.

```
Feature: US-02 Comic Detail View & Quick Chapter Actions (Maps to FR-10,
   UC-02, S04)
  As a reader, I want to inspect comic metadata and trigger Read First,
     Read Latest, Favorite, or Download
  So that I can jump into reading or bookmark the series immediately.

  Scenario: Launching Read First or Read Latest and bookmarking to
     Favorites
    Given the user opens "Frieren: Beyond Journey's End" on "S04 Manga
       Detail"
    When the user taps "Read First" (or "Read Latest")
    Then the system initiates preloading ("S10") for Chapter 1 (or the
       latest chapter)
    When the user taps the "Favorite" button
```

```
    Then the comic is saved to local SQLite "bookmarks" and synced to
       Supabase Cloud

Feature: US-03 Full-Screen Dual-Mode Reader & Focus Mode (Maps to FR-02,
   FR-03, FR-12, UC-03, S05)
  As an immersed reader, I want a 100% full-screen canvas with Focus Mode
     and Vertical/Horizontal toggle
  So that I can read comfortably at 60 FPS without visual clutter.

  Scenario: Toggling Focus Mode and switching between Vertical Webtoon
     and Horizontal Manga modes
    Given the user is reading Chapter 24 on "S05 Full Screen Reader"
    When the user taps the Focus Mode icon on the top toolbar
    Then the top toolbar hides completely leaving 100% unobstructed manga
       artwork
    When the user taps the canvas to restore the toolbar and taps the
       Reader Mode Toggle icon
    Then the reader switches smoothly between Vertical Scroll and
       Horizontal Page Flip

Feature: US-04 Genre-Matched Instrumental Ambient Audio (Maps to FR-05,
   FR-09, UC-04, S06)
  As a late-night reader, I want instrumental music automatically matched
     to the comic's genre
  So that I can enjoy an atmospheric reading session without running
     external music apps.

  Scenario: Controlling genre-matched instrumental playback in the Audio
     Sheet
    Given the user is reading a "Fantasy" comic on "S05" and taps the
       Music icon
    When the "S06 Audio Sheet" slides up
    Then the system streams the recommended Fantasy instrumental track
       ("Winds of Aureole")
    And allows pausing/resuming, sliding volume, or tapping another track
       in the genre list

Feature: US-05 Full-Chapter Pre-buffering & Page-Level Progress Sync
   (Maps to FR-04, UC-05, S10, S02)
  As a reader, I want all pages of a chapter preloaded before reading and
     my exact page saved automatically
  So that scrolling never stutters and I can resume instantly from the
     Home screen.

  Scenario: Pre-buffering 100% of chapter pages and auto-saving page
     progress
    Given the user selects a 42-page chapter to read
    When "S10 Full Chapter Preload" runs
    Then the progress bar advances to "42 / 42 pages ready (100%)" before
       opening "S05"
    When the user scrolls to Page 18 and exits the app
```

```
    Then "last_page_read = 18" (43%) is saved to SQLite within 50ms and
       synced to the "S02" Continue card
```

Listing 3: US-06, US-07 & US-08: Offline Downloads, Network Fallback, Library, and Settings.

```
Feature: US-06 Offline Chapter Download & Network Loss Fallback (Maps to
   FR-06, FR-13, UC-06, S09, S11)
  As a commuting reader, I want to download chapters for offline reading
     and be guided to my offline library when connectivity drops
  So that weak 4G never interrupts my entertainment.

  Scenario: Downloading a chapter with real-time progress and handling
     offline errors gracefully
    Given the user taps "Download" on Chapter 24 in "S04"
    Then "S11 Download in Progress" displays real-time progress ("68% -
       24.5 MB / 36 MB") until marked "Saved"
    Given the device subsequently loses all Internet connectivity
    When the user taps an uncached chapter
    Then the system transitions within 300ms to "S09 Offline Error State"
       ("Oops! No internet connection")
    And tapping "Go to Offline Library" routes the user directly to "S07
       Library"

Feature: US-07 & US-08 Personal Library & System Settings (Maps to FR-07,
   FR-14, UC-07, UC-08, S07, S08)
  As a user, I want a clean two-button Library and configurable Theme,
     Language, and Cache settings
  So that I can manage my collection and device storage effortlessly.

  Scenario: Navigating the two-button Library and switching Appearance
     Theme
    Given the user opens "S07 Library"
    Then two primary cards ("Favorites" and "Offline Reading") are
       displayed with item counts and MB size
    Given the user navigates to "S08 Profile & Settings" and selects
       "Dark" under Appearance
    Then the application theme updates immediately and persists in user
       preferences
```

3 Non-Functional Requirements & Mobile Constraints

3.1 SMART Non-Functional Design Philosophy

Delivering functional features alone is insufficient for a media-intensive mobile entertainment application such as **Naomi**. Mobile devices operate under strict physical and operating-system constraints: **finite RAM budgets, limited battery capacity, fluctuating cellular 4G/5G connectivity during transit, and stringent OS sandboxing/permission models**. Consequently, every Non-Functional Requirement (NFR) in this specification is engineered according to the **SMART** principle:

- **Specific & Mobile-Relevant:** Directly tied to concrete mobile runtime mechanisms (GPU frame rendering, Android Keystore/iOS Keychain, OS AudioSession focus, SQLite indexing, and System Photo Picker).
- **Measurable:** 100% of requirements define explicit **quantitative metrics, numerical acceptance thresholds, and physical units** (ms, s, FPS, MB, %/hour), eliminating vague statements such as *"the app must be fast"*.
- **Achievable, Relevant & Time-Bound:** Verified through industry-standard profiling tools (*Flutter DevTools, Android Studio Memory/Network Profiler, Android Vitals, and Battery Historian*) during Assignment 02 acceptance testing.

3.2 Master Non-Functional Requirements Matrix

Table 7 consolidates all 18 quantitative Non-Functional Requirements (`NFR-01` to `NFR-18`) structured across the six core mobile engineering pillars required by the course rubric.

Table 7: Master Non-Functional Requirements Matrix (`NFR-01` to `NFR-18`) with SMART thresholds and verification tools.

NFR ID	Pillar	Technical Criterion	Quantitative Metric	Threshold	Verification Tool
NFR-01	**Performance**	Cold Start Time	Elapsed time from app icon tap to first rendered Home/Sign-In frame	≤ 2.0 s	Android Vitals / Flutter DevTools
NFR-02	**Performance**	Warm / Hot Start Time	Time to resume app from background to interactive foreground state	≤ 0.8 s	Flutter DevTools Performance
NFR-03	**Performance**	Reader Scrolling Frame Rate	Sustained rendering rate while scrolling a 50-page Webtoon chapter	≥ 60 FPS (≤ 16.6 ms)	Flutter Performance Overlay
NFR-04	**Performance**	Chapter Preload Initiation Latency	Time to resolve MangaDex@Home CDN node and begin pre-buffering on `S10` over 4G	≤ 1.5 s	Network Profiler / Stopwatch

Continued on next page

Table 7 – continued from previous page

NFR ID	Pillar	Technical Criterion	Quantitative Metric	Threshold	Verification Tool
NFR-05	**Performance**	Authentication API Response Time	Round-trip latency for Supabase Auth JWT token issuance	$\leq$ 1.0 s	Postman / Network Profiler
NFR-06	**Memory**	Peak Runtime RAM Footprint	Maximum heap + GPU texture memory consumed during long-chapter reading	$\leq$ 250 MB	Android Studio Memory Profiler
NFR-07	**Offline & Storage**	Dynamic Image Disk Cache Cap	Maximum storage footprint of temporary network image cache before eviction	$\leq$ 500 MB (LRU policy)	App File Storage Inspector
NFR-08	**Offline & Storage**	Offline Mode Transition Latency	Time to detect network loss, suppress crash, and route to `S09`/SQLite	$\leq$ 300 ms	ConnectivityPlus Listener Test
NFR-09	**Offline & Storage**	Local SQLite Query / Write Speed	Execution time for reading/updating `reading_history` or `bookmarks`	$\leq$ 50 ms / query	SQLite Benchmark / Drift Log
NFR-10	**Security**	Hardware-Backed Token Storage	Cryptographic algorithm protecting stored JWT access and refresh tokens	**AES-256 GCM** (Keystore/KC)	Root/Sandbox Storage Audit
NFR-11	**Security**	Transport Layer Encryption	Share of client-server traffic encrypted with modern TLS	**100% HTTPS / TLS 1.3**	Charles Proxy / Manifest Audit
NFR-12	**Permissions**	Principle of Least Privilege	Number of dangerous OS runtime permissions requested (GPS, Camera, Storage, Mic)	**0 Dangerous Permissions**	`AndroidManifest` & Play Policy
NFR-13	**Permissions**	Graceful Permission Denial Handling	Application stability when user denies optional `POST_NOTIFICATIONS` (API 33+)	**0 crashes**; fallback to in-app toast (`S11`)	Manual UI Acceptance Test
NFR-14	**Platform**	OS Version Compatibility	Minimum and target mobile OS versions supported	**Android 8.0+** (API 26–34), **iOS 13.0+**	Firebase Test Lab / Real Devices
NFR-15	**Platform**	Multi-Aspect Ratio Responsiveness	UI layout integrity across 16:9 to 21:9 screens, notches, and gesture bars	**100% within SafeArea**	Flutter Layout Inspector
NFR-16	**Battery**	Active Reading + Audio Battery Drain	Battery percentage consumed during 60 minutes of continuous reading + ambient audio	$\leq$ 8% / hour	Android Battery Historian
NFR-17	**Battery**	Audio Focus Interruption Latency	Time to pause instrumental playback upon incoming call or screen lock	$\leq$ 100 ms	`AudioSession` Focus Test
NFR-18	**Network**	Cellular Bandwidth Reduction	Image payload size reduction when switching to MangaDex `dataSaver` on 4G/5G	**40%–60%** reduction	Network Traffic Monitor

3.3 Detailed Technical Justification across the Six Mobile Pillars

3.3.1 Pillar 1: Performance, Frame Budget & Preload Latency (NFR-01 to NFR-06)

1. **Application Startup (`NFR-01` $\leq$ 2.0 s, `NFR-02` $\leq$ 0.8 s):** Cold start is optimised by deferring non-critical service initialisation (*lazy dependency injection*) until after the first frame renders and reading cached Home metadata from local SQLite Drift immediately before background cloud synchronisation begins.
2. **Sustained 60 FPS Frame Budget (`NFR-03` $\leq$ 16.6 ms/frame) & RAM Ceiling (`NFR-06` $\leq$ 250 MB):** A standard 60 Hz display requires every UI and raster frame to complete within 1000 ms/60 $\approx$ 16.6 ms. To prevent frame drops and Out-Of-Memory crashes on mid-range 3–4 GB RAM devices, **Naomi** combines **Full-Chapter Pre-buffering (`S10`)** on disk with a **Virtualised Viewport List (`ListView.builder` / `SliverList`)** in GPU memory: only the pages currently visible inside the viewport plus a $\pm$2-page lookahead buffer are decoded into active GPU textures, while off-screen page textures are immediately recycled.
3. **Chapter Preload Initiation (`NFR-04` $\leq$ 1.5 s) & Auth Latency (`NFR-05` $\leq$ 1.0 s):** Resolving the nearest `MangaDex@Home` CDN edge node (`API 03`) and initiating concurrent page pre-buffering on `S10` completes within 1.5 s on standard 4G, while Supabase JWT authentication (`API 04`) completes within 1.0 s.

3.3.2 Pillar 2: Offline Behaviour, Bounded Caching & Local Persistence (NFR-07 to NFR-09)

1. **Bounded 500 MB LRU Disk Cache (`NFR-07`):** Temporary online page caches (*Tier 4 Storage*) are strictly capped at a hard ceiling of 500 MB (soft warning at 400 MB). When the cache reaches 500 MB, a Least Recently Used (**LRU**) eviction worker prunes the oldest accessed files down to 350 MB. User-initiated *Offline Downloads* (*Tier 3 Storage*) reside in a separate protected directory (`ApplicationDocumentsDirectory`) and are never auto-evicted.
2. **Sub-300 ms Offline Fallback (`NFR-08`) & Sub-50 ms SQLite I/O (`NFR-09`):** By subscribing to `ConnectivityPlus` network state streams, **Naomi** detects sudden connectivity loss (e.g., entering an underground tunnel or Airplane Mode) within $\leq$ 300 ms and routes uncached chapter requests cleanly to `S09` (*Offline Error State*) without unhandled exceptions. Foreign-key indexing on `(user_id, chapter_id)` and `(user_id, comic_id)` guarantees asynchronous SQLite reads/writes under 50 ms.

3.3.3 Pillar 3: Mobile Security & Cryptographic Privacy (NFR-10, NFR-11)

1. **Hardware-Backed Token Encryption (`NFR-10`):** Sensitive credentials (`access_token`, `refresh_token`) are never stored in plain-text `SharedPreferences`. All tokens are persisted exclusively via `FlutterSecureStorage`, backed by **AES-256 GCM in Android Keystore** and the **iOS Keychain (Secure Enclave)**.
2. **Mandatory TLS 1.3 & Cleartext Blocking (`NFR-11`):** 100% of network communication with Supabase and MangaDex is enforced over **HTTPS / TLS 1.3**. Plain-text HTTP traffic is explicitly disabled at the OS manifest level (`android:usesCleartextTraffic="false"`). API keys are injected at compile time via `-dart-define` and protected in release builds with R8/ProGuard code obfuscation.

3.3.4 Pillar 4: System Permissions & Zero-Storage-Permission Visual Search (NFR-12, NFR-13)

1. **Zero Dangerous Permissions (`NFR-12`):** Adhering strictly to the *Principle of Least Privilege*, **Naomi** requests **zero dangerous runtime permissions**: no GPS location, no Contacts, no Camera hardware access, no Microphone, and no broad External Storage read permissions. Only standard install-time network permissions (`INTERNET`, `ACCESS_NETWORK_STATE`) and optional Android 13+ `POST_NOTIFICATIONS` (for download progress banners, `NFR-13`) are declared.
2. **System Photo Picker Architecture for Visual Search (`FR-11`):** Rather than requesting intrusive `READ_MEDIA_IMAGES` or `READ_EXTERNAL_STORAGE` permissions to scan user screenshots, **Naomi** invokes the native OS **System Photo Picker** (`AndroidX PickVisualMedia` / iOS `PHPickerViewController`), which grants a temporary, single-file read-only URI only for the specific screenshot selected by the user.

3.3.5 Pillar 5 & Pillar 6: Platform Support, Battery & Network Efficiency (NFR-14 to NFR-18)

1. **Broad OS Coverage & Responsive SafeArea (`NFR-14`, `NFR-15`):** Targeting **Android 8.0+ (API 26 to API 34)** covers $> 95.4\%$ of active Android devices globally alongside **iOS 13.0+**. Every screen is wrapped in `SafeArea` with an 8 px spatial grid to adapt cleanly across 16:9 through 21:9 aspect ratios, camera notches, and gesture navigation bars.
2. **Battery Efficiency ($\leq$ 8%/hour, `NFR-16`) & Audio Focus (`NFR-17`):** The AMOLED dark reader canvas (`#111116`) reduces OLED pixel power draw by up to 30%, while `just_audio` delegates MP3 decoding to native hardware codecs. When an incoming phone call, alarm, or screen lock occurs, `AudioSession` relinquishes audio focus and pauses playback within ≤ 100 ms.

3. **Cellular Data Saver (`NFR-18`):** When `ConnectivityPlus` detects a metered cellular connection (4G/5G) instead of Wi-Fi, the repository automatically requests compressed `dataSaver` WebP/JPG assets from `MangaDex@Home` (`API 03`), cutting bandwidth consumption by 40%–60%.

3.4 NFR Quality Assurance & Profiling Plan

Table 8 defines the concrete profiling protocol to be executed in Assignment 02 to verify compliance with all NFR thresholds.

Table 8: Quantitative QA verification and profiling protocol for Assignment 02 acceptance.

Verification Target	Profiling Instrument	Step-by-Step Acceptance Test Procedure
Frame Rate (≥ 60 FPS) & Preload (`NFR-03, NFR-04`)	Flutter Performance Overlay & DevTools Timeline	Enable Performance Overlay in profile mode; preload and scroll rapidly through 10 consecutive 40-page chapters; verify UI and Raster threads remain below the 16.6 ms frame budget line.
RAM Ceiling (≤ 250 MB) & Texture Recycling (`NFR-06`)	Android Studio Memory Profiler	Attach a physical Android device; read continuously for 30 minutes across multiple chapters; verify total memory stays ≤ 250 MB with zero monotonic texture leaks.
Offline Survival (≤ 300 ms) & SQLite Speed (`NFR-08, NFR-09`)	Airplane Mode Toggle & Drift Query Logger	Toggle Airplane Mode mid-session; verify uncached chapters route to `S09` within 300 ms, downloaded chapters open from SQLite, and page-progress writes complete in ≤ 50 ms.
Battery Drain ($\leq 8\%$/hr) & Security (`NFR-10..12, NFR-16`)	Android Battery Historian & Manifest/Keystore Audit	Measure 60-minute reading + ambient audio session via Battery Historian; inspect `AndroidManifest.xml` to confirm zero dangerous permissions and `usesCleartextTraffic="false"`.

4 System Architecture Design

4.1 High-Level Distributed Multi-Tier Architecture

The **Naomi** platform is engineered as a **Distributed 4-Tier Hybrid Offline-First Architecture** (Figure 3), decoupling the mobile presentation client from cloud identity/synchronisation services, open global comic content delivery, and on-device relational/cryptographic persistence:

1. **Client Tier (Naomi Flutter Mobile Application):** A single cross-platform Dart/Flutter codebase compiled ahead-of-time (AOT) to native ARM64 machine code via the **Impeller / Skia** rendering engine, structured internally into Presentation, Domain, and Data layers.
2. **App Backend Tier (Supabase BaaS Cloud):** Provides enterprise JWT authentication (`Supabase Auth`), a cloud `PostgreSQL` relational database protected by Row-Level Security (**RLS**) for user profiles, bookmarks, reading history, and genre-audio mappings, and `Supabase Storage` buckets serving streaming MP3 instrumental tracks.
3. **External Content Tier (MangaDex Ecosystem):** Supplies global comic and chapter metadata via **MangaDex REST API v5** (5 req/s rate-limited) and delivers high-resolution or `dataSaver` WebP/JPG page assets through the distributed **MangaDex@Home CDN** network.
4. **Local Persistence Tier (On-Device Storage):** Guarantees 100% offline availability via type-safe **Drift / SQLite** relational tables, hardware-backed **FlutterSecureStorage** (Android Keystore / iOS Keychain AES-256), sandboxed offline chapter files (`ApplicationDocumentsDirectory`), and a bounded 500 MB LRU disk cache.

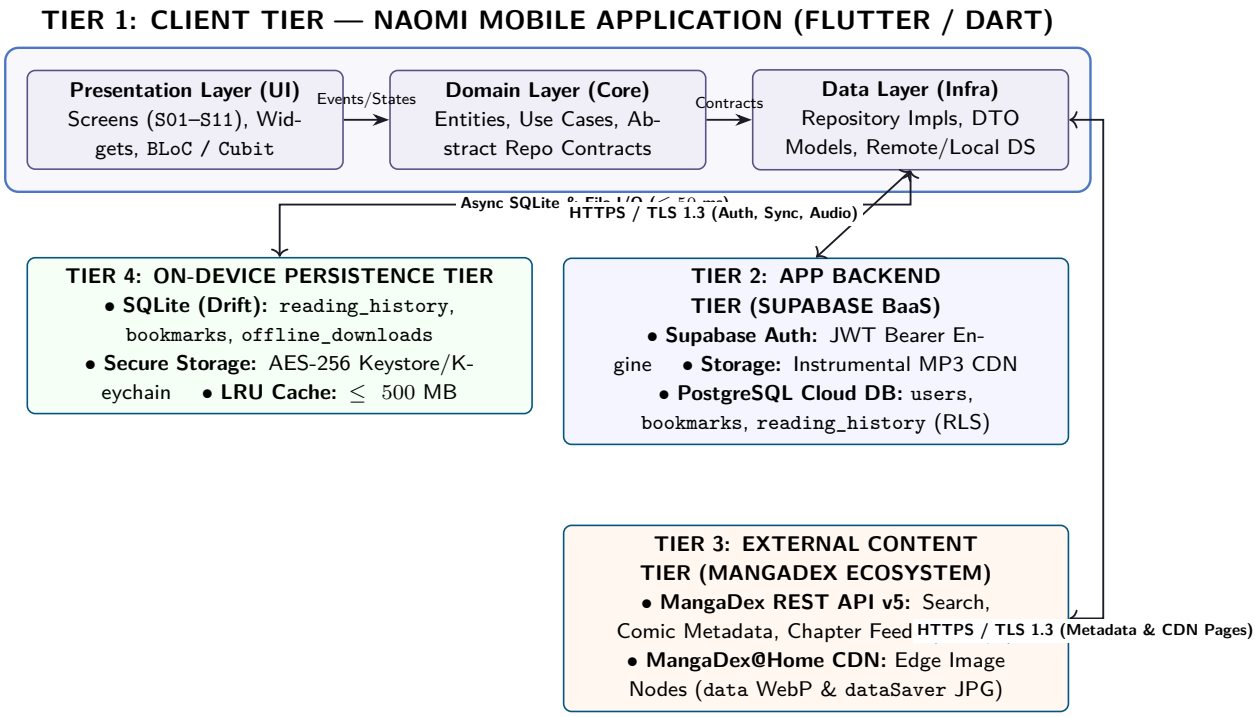


Figure 3: C4 Container & Multi-Tier System Architecture Diagram of the Naomi Ecosystem.

4.2 In-App Clean Architecture & BLoC State Management

To satisfy Rubric Criterion 1.4 and 2.2 (*"Follows the designed architecture, clear layering, sound state management"*), the internal mobile codebase enforces Uncle Bob's **Clean Architecture** combined with the unidirectional **BLoC (Business Logic Component)** pattern (Figure 4):

- **Presentation Layer (`lib/presentation/`):** Contains declarative Flutter Screens (`S01`–`S11`), reusable UI widgets, and `BLoC`/`Cubit` state controllers (`AuthBloc`, `HomeBloc`, `DiscoverBloc`, `ComicDetailBloc`, `ReaderBloc`, `AudioCubit`, `DownloadBloc`, `LibraryBloc`, `SettingsCubit`, `ConnectivityCubit`). UI widgets never invoke network or database APIs directly; they only dispatch immutable `Events` and rebuild reactively on emitted `States` (`Initial`, `Loading`, `Loaded`, `Empty`, `Error`).
- **Domain Layer (`lib/domain/` – Pure Dart Core):** Houses immutable business `Entities` (`ComicEntity`, `ChapterEntity`, `PageEntity`, `AmbientTrackEntity`, `UserProfileEntity`), single-responsibility `UseCases` (`SignInUseCase`, `SearchComicsUseCase`, `GetChapterContentUseCase`, `SaveReadingProgressUseCase`, `GetGenreAudioUseCase`), and abstract `Repository Interfaces` (`IComicRepository`, `IAuthRepository`). In strict compliance with **The Dependency Rule**, the Domain layer has zero imports from Flutter UI, `dio`, `drift`, or `supabase_flutter`.
- **Data Layer (`lib/data/` – Infrastructure):** Implements the domain contracts (`ComicRepositoryImpl`, `AuthRepositoryImpl`), serialises JSON/SQLite `DTO Models`, and orchestrates Remote Data Sources (`MangaDexApiClient`, `SupabaseApiClient`) alongside

Local Data Sources (`DriftAppDatabase`, `SecureStorageManager`, `LocalFileStorage`).

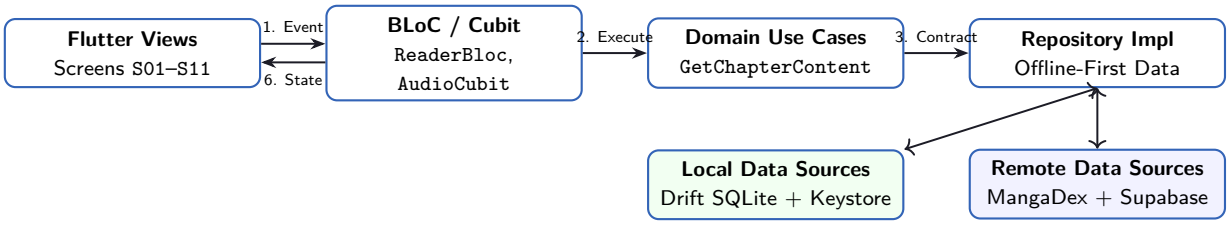


Figure 4: Unidirectional data and state flow across Clean Architecture layers (`UI` ↔ `BLoC` ↔ `UseCase` ↔ `Repository`).

4.3 UML Sequence Diagrams for Core System Flows

Figures 5, 6, and 7 model the exact runtime message choreography across architectural components for the three primary operational flows of **Naomi**.

4.3.1 Flow 1: Full-Screen Preloaded Reading with Genre Ambient Audio (UC-03, UC-04, UC-05)

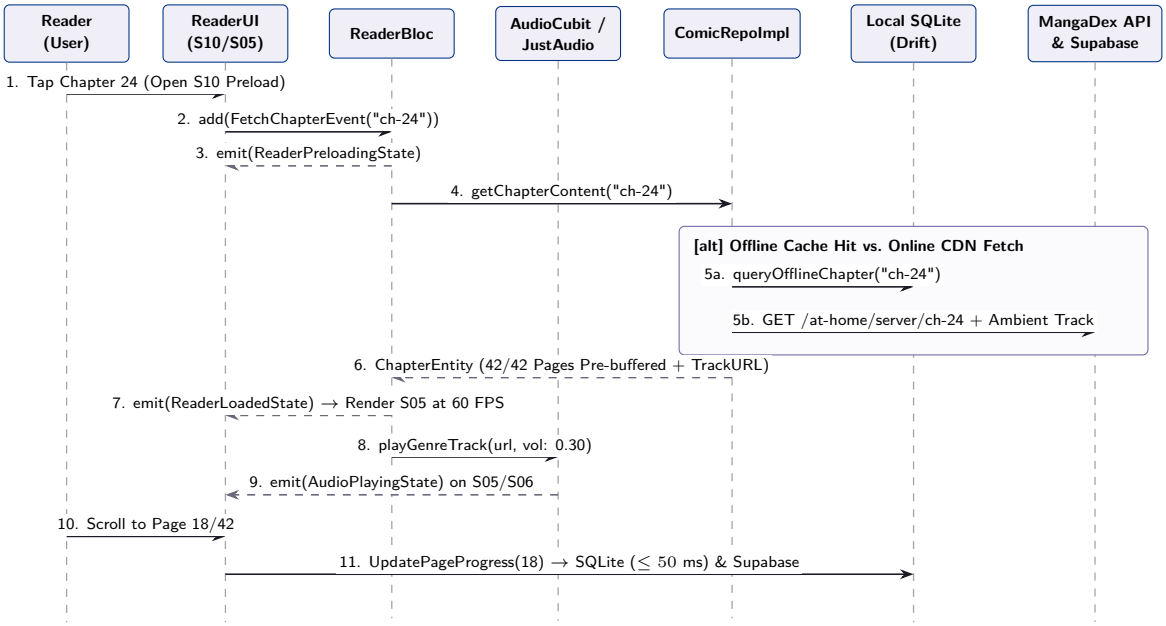


Figure 5: UML Sequence Diagram for Flow 1: Full-Chapter Preload (`S10`), 60 FPS Reading (`S05`), Genre Ambient Audio (`S06`), and Page Progress Sync.

4.3.2 Flow 2: Offline Chapter Download & Sandboxed Persistence (UC-06)

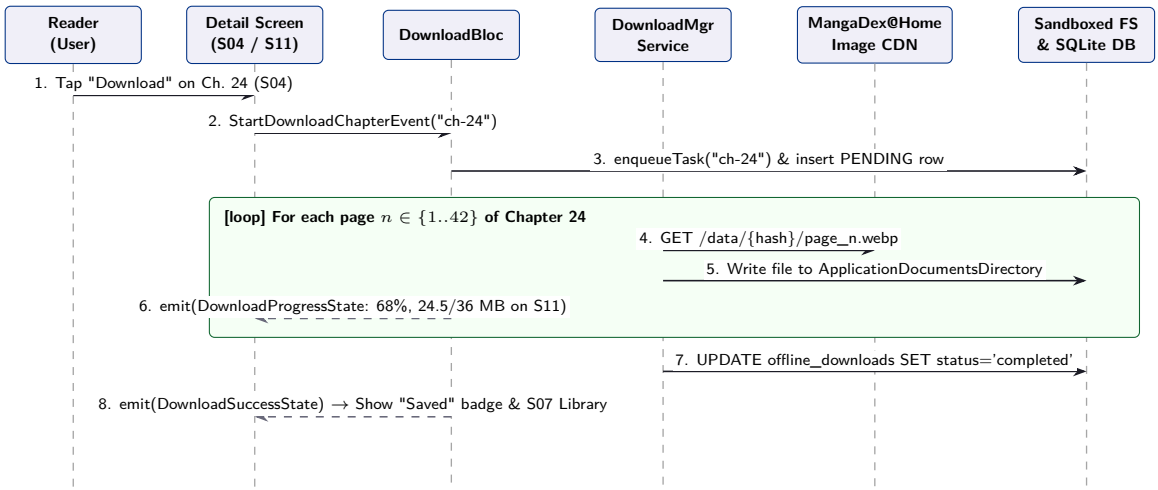


Figure 6: UML Sequence Diagram for Flow 2: Sequential Chapter Page Download (`S11`) and Local SQLite Registration.

4.3.3 Flow 3: User Authentication & Hardware-Backed Auto-Login (UC-00)

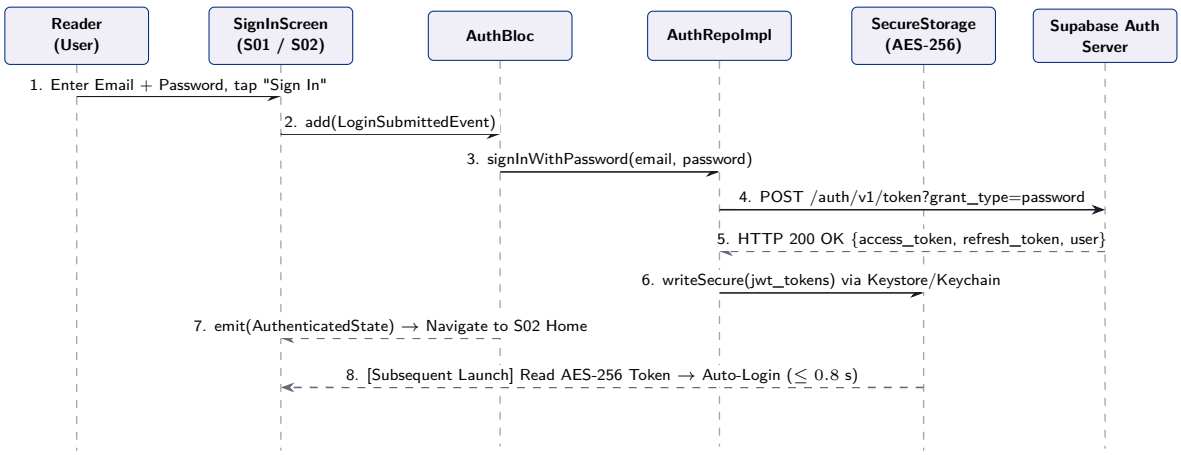


Figure 7: UML Sequence Diagram for Flow 3: Supabase JWT Authentication (`S01`) and Hardware-Backed Keystore Persistence.

4.4 Justified Technology Choices

To fulfil Rubric Criterion 1.4 (*"justified technology choices"*), Table 9 presents a rigorous comparative trade-off evaluation across the four foundational architectural decisions of **Naomi**.

Table 9: Comparative architectural trade-off analysis and justification of selected technologies.

Architectural Layer	Selected Technology	Evaluated Alternatives	Technical Rationale & Mobile Engineering Justification
Cross-Platform Mobile Framework	**Flutter 3.x (Dart, Impeller / Skia)**	React Native (JS Bridge), Native Kotlin / Swift	**Eliminates bridge bottlenecks during Webtoon scrolling:** Flutter compiles AOT to native ARM64 code and renders every pixel directly onto its own Impeller/Skia GPU canvas, sustaining ≥ 60 FPS (`NFR-03`) across 50-page vertical strips while halving development effort compared to dual Kotlin/Swift codebases.
In-App Architecture & State Management	**Clean Architecture + `flutter_bloc`**	MVC / Basic MVVM, Vanilla `Provider` / `setState`	**Strict separation of concerns & deterministic unit testing:** Unidirectional `Event` $\rightarrow$ `BLoC` $\rightarrow$ `State` streams prevent "Fat Widget/Controller" anti-patterns and allow 100% isolated unit testing of domain use cases and BLoC transitions in Assignment 02 via `bloc_test`.
Cloud Backend & Auth Platform	**Supabase BaaS (PostgreSQL + Auth + Storage)**	Custom Node.js/Express + MongoDB, Firebase	**Relational integrity + zero-downtime cloud infrastructure:** Unlike NoSQL Firestore/MongoDB, Supabase provides a true **PostgreSQL** engine enforcing 3NF/BCNF foreign keys and database-level **Row-Level Security (RLS)**, paired with built-in JWT Auth and CDN Buckets for streaming MP3 ambient audio.
On-Device Local Database	**SQLite via `Drift` (Type-Safe ORM)**	`SharedPreferences`, NoSQL `Hive`	**Relational hierarchy & sub-50 ms indexed queries:** Comic data is inherently relational (`comics` 1:N `chapters` 1:N `pages`). SQLite supports foreign-key `ON DELETE CASCADE` and composite indexes on `(user_id, chapter_id)`, achieving ≤ 50 ms query times (`NFR-09`) without blocking the UI thread.

5 Data & API Design

5.1 Hybrid Offline-First Data Architecture & Data Dictionary

The data persistence layer of **Naomi** follows a **Hybrid Offline-First** paradigm spanning three synchronised storage domains: (1) **Supabase Cloud PostgreSQL** as the single source of truth for user identity, cloud bookmarks, reading history, and genre-audio mappings; (2) **MangaDex REST API v5 & MangaDex@Home CDN** as the global comic metadata and page-image provider; and (3) **On-Device SQLite (Drift) + Sandboxed File Storage** for instant local reads and full offline operation. Table 10 summarises the 10 normalised relational tables.

Table 10: System Data Dictionary across the 10 normalised tables of the Naomi database schema.

#	Table Name	Storage Location	Primary Purpose & Entity Role	FR Mapping
1	`users`	Supabase Cloud	User profile linked 1:1 to `auth.users(id)`, display name, avatar, and JSONB UI preferences.	`FR-00`, `FR-14`
2	`comics`	Cloud & Local Cache	Comic series metadata (title, alt_title, author, artist, status, cover_url, default ambient track).	`FR-01`, `FR-08`, `FR-10`
3	`genres`	Cloud & Local Cache	Canonical genre catalogue (`action`, `fantasy`, `romance`, `slice_of_life`, etc.).	`FR-01`, `FR-05`
4	`comic_genres`	Cloud & Local Cache	Associative M:N join table linking `comics` and `genres` via composite PK.	`FR-01`, `FR-05`
5	`chapters`	Cloud & Local Cache	Individual chapter metadata (chapter_number, volume, title, page_count, optional audio override).	`FR-02`, `FR-04`, `FR-10`
6	`pages`	Cloud & Local Cache	**Weak Entity** representing ordered pages within a chapter, identified by `(chapter_id, page_number)`.	`FR-02`, `FR-03`
7	`ambient_tracks`	Supabase DB/Storage	Instrumental background tracks categorised by genre with streaming MP3 URLs and loop metadata.	`FR-05`, `FR-09`
8	`reading_history`	Cloud & Local SQLite	Exact page-level progress (`last_page_read`, `read_percentage`, `is_synced`); strictly 3NF.	`FR-04`, `FR-08`
9	`bookmarks`	Cloud & Local SQLite	User's personal bookmarked series (`is_favorite`, `reading_status`) synced across devices.	`FR-07`, `FR-10`
10	`offline_downloads`	Local SQLite Only	Tracks local chapter download status, byte size, and relative directory path in sandboxed storage; strictly 3NF.	`FR-06`, `FR-07`, `FR-13`

5.2 Conceptual & Physical Entity-Relationship Diagrams

Following standard database design methodology (*Elmasri & Navathe, Fundamentals of Database Systems, 7th Ed., Chapters 2–3*), Figure 8 presents the **Conceptual Entity-Relationship Diagram in Chen Notation** (highlighting strong vs. weak entities, identifying relationships, total participation double lines, and partial key dashed underlines), while Figure 9 presents the mapped **Physical Relational Schema in Crow's Foot Notation**.

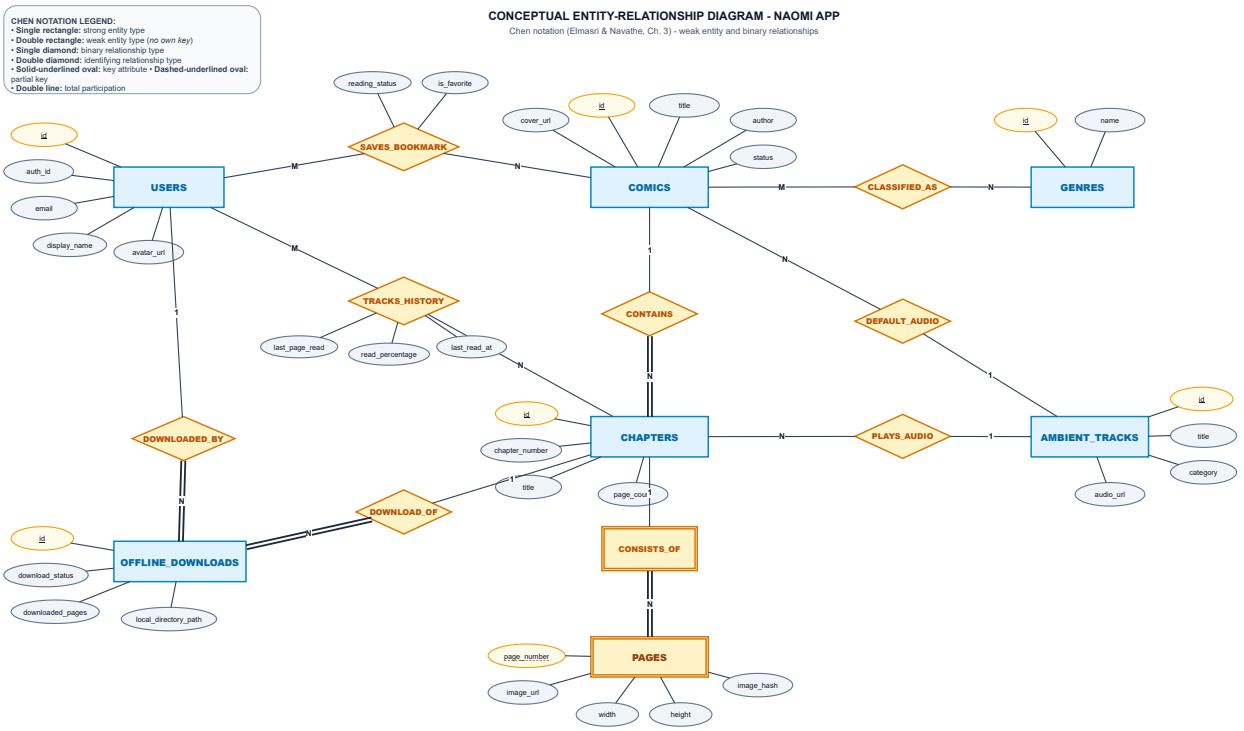


Figure 8: Conceptual Entity-Relationship Diagram (Chen Notation) illustrating strong entities, the weak entity `PAGES` with partial key `page_number`, and binary relationships.

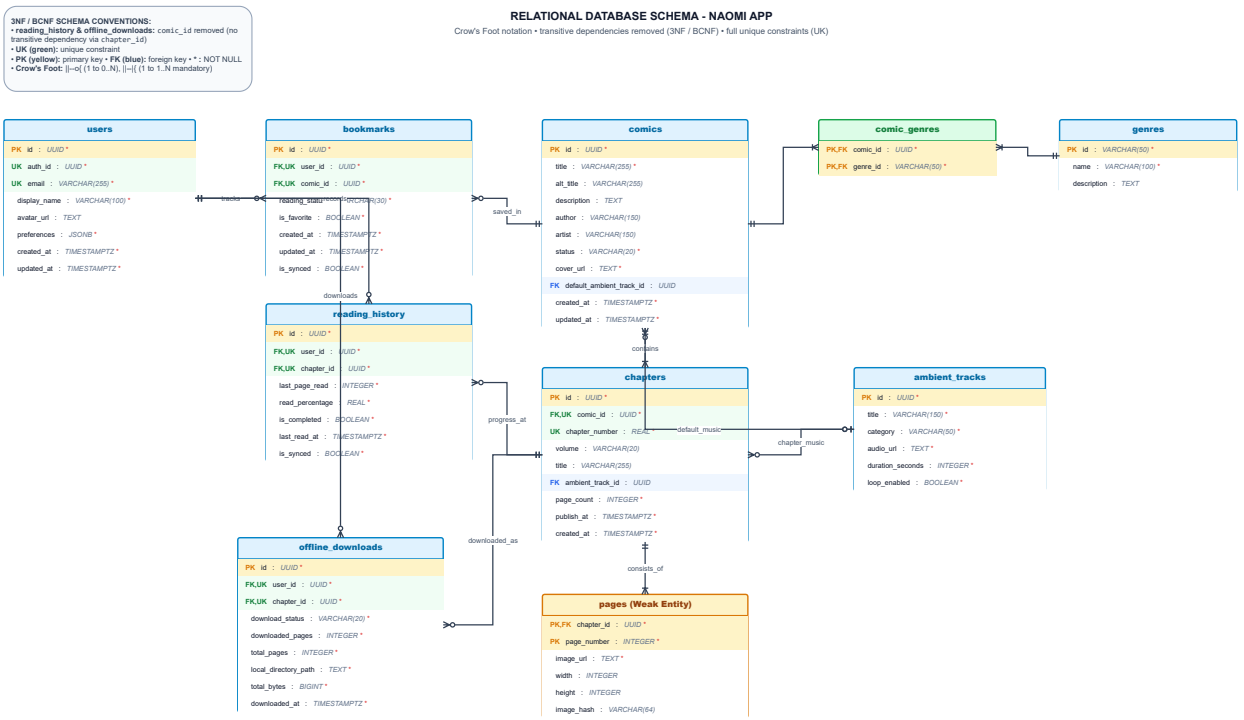


Figure 9: Physical Relational Database Schema (Crow's Foot Notation) enforcing 3NF/BCNF, Primary Keys (PK), Foreign Keys (FK), and Unique Constraints (UK).

5.3 Detailed Relational Schema, 3NF/BCNF Normalisation Proof & RLS Security

5.3.1 Relational Schema Specifications (10 Tables)

- **users:** id UUID PK, auth_id UUID UNIQUE NOT NULL, email VARCHAR(255) UNIQUE NOT NULL, display_name VARCHAR(100) NOT NULL, avatar_url TEXT, preferences JSONB DEFAULT '{"scroll_direction": "vertical", "ambient_auto_play": true, "ambient_volume":0.3}', created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ.
- **comics:** id UUID PK, title VARCHAR(255) NOT NULL, alt_title VARCHAR(255), description TEXT, author VARCHAR(150), artist VARCHAR(150), status VARCHAR(20) CHECK (status IN ('ongoing','completed','hiatus','cancelled')), cover_url TEXT NOT NULL, default_ambient_track_id UUID FK → ambient_tracks(id) ON DELETE SET NULL, created_at, updated_at.
- **genres & comic_genres:** genres(id VARCHAR(50) PK, name VARCHAR(100) NOT NULL, description TEXT); comic_genres(comic_id UUID FK, genre_id VARCHAR(50) FK, PRIMARY KEY (comic_id, genre_id)).
- **chapters:** id UUID PK, comic_id UUID NOT NULL FK → comics(id) ON DELETE CASCADE, chapter_number REAL NOT NULL, volume VARCHAR(20), title VARCHAR(255), ambient_track_id UUID FK → ambient_tracks(id) ON DELETE SET NULL, page_count INT NOT NULL, publish_at TIMESTAMPTZ, created_at TIMESTAMPTZ, with UNIQUE(comic_id, chapter_number).
- **pages (Weak Entity Mapping):** chapter_id UUID NOT NULL FK → chapters(id) ON DELETE CASCADE, page_number INT NOT NULL, image_url TEXT NOT NULL, width INT, height INT, image_hash VARCHAR(64), with composite PRIMARY KEY (chapter_id, page_number).
- **ambient_tracks:** id UUID PK, title VARCHAR(150) NOT NULL, category VARCHAR(50) NOT NULL, audio_url TEXT NOT NULL, duration_seconds INT NOT NULL, loop_enabled BOOLEAN DEFAULT TRUE.
- **reading_history (3NF/BCNF):** id UUID PK, user_id UUID NOT NULL FK → users(id) ON DELETE CASCADE, chapter_id UUID NOT NULL FK → chapters(id) ON DELETE CASCADE, last_page_read INT NOT NULL DEFAULT 1, read_percentage REAL NOT NULL DEFAULT 0.0, is_completed BOOLEAN DEFAULT FALSE, last_read_at TIMESTAMPTZ, is_synced BOOLEAN DEFAULT TRUE, with UNIQUE(user_id, chapter_id).
- **bookmarks:** id UUID PK, user_id UUID NOT NULL FK → users(id) ON DELETE CASCADE, comic_id UUID NOT NULL FK → comics(id) ON DELETE CASCADE, reading_status VARCHAR(30) DEFAULT 'reading', is_favorite BOOLEAN DEFAULT FALSE, created_at, updated_at, is_synced BOOLEAN, with UNIQUE(user_id, comic_id).

- **offline_downloads (3NF/BCNF):** id UUID PK, user_id UUID NOT NULL FK → users(id) ON DELETE CASCADE, chapter_id UUID NOT NULL FK → chapters(id) ON DELETE CASCADE, download_status VARCHAR(20) CHECK (download_status IN ('pending','downloading', 'completed','failed')), downloaded_pages INT, total_pages INT NOT NULL, local_directory_path TEXT NOT NULL, total_bytes BIGINT, downloaded_at TIMESTAMPTZ, with UNIQUE(user_id, chapter_id).

5.3.2 Formal 3NF & Boyce-Codd Normal Form (BCNF) Proof

1. **First Normal Form (1NF):** Every attribute across all 10 relations holds atomic values. Multi-valued genre tags are decomposed into the associative relation comic_genres(comic_id, genre_id).
2. **Second Normal Form (2NF):** All single-attribute UUID primary keys are trivially free of partial dependencies. For composite-key tables—comic_genres(comic_id, genre_id) (no non-key attributes) and the weak entity pages(chapter_id, page_number)—every non-key attribute (image_url, width, height, image_hash) depends irreducibly on both chapter_id and page_number.
3. **Third Normal Form (3NF) & BCNF:** Because every chapter belongs to exactly one comic (chapter_id → comic_id), storing comic_id inside reading_history or offline_downloads would introduce the transitive functional dependency id → chapter_id → comic_id. Our schema **explicitly eliminates comic_id from both reading_history and offline_downloads**, resolving parent comic metadata via an indexed join on chapters(comic_id). Similarly, audio metadata is factored out into ambient_tracks. Consequently, for every non-trivial functional dependency $X \to Y$ across all 10 tables, X is a superkey, proving strict compliance with **BCNF** (*Elmasri & Navathe, Ch. 5*).

5.3.3 Database Security & Row-Level Security (RLS) Policies

To enforce tenant isolation at the database engine level (NFR-10, NFR-11), Supabase PostgreSQL Row-Level Security is enabled across all user-data tables:

Listing 4: Supabase PostgreSQL Row-Level Security (RLS) Policies for User Isolation.

```
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE reading_history ENABLE ROW LEVEL SECURITY;
ALTER TABLE bookmarks ENABLE ROW LEVEL SECURITY;

-- 1. USERS: Users can only SELECT, INSERT, or UPDATE their own profile
    row
CREATE POLICY "Users view own profile" ON users FOR SELECT USING (auth.uid
    () = auth_id);
CREATE POLICY "Users insert own profile" ON users FOR INSERT WITH CHECK (
    auth.uid() = auth_id);
```

```
CREATE POLICY "Users update own profile" ON users FOR UPDATE USING (auth.uid() = auth_id);

-- 2. READING_HISTORY & BOOKMARKS: Strict per-user isolation via auth.uid()
CREATE POLICY "Users manage own reading history" ON reading_history FOR ALL
  USING (auth.uid() = (SELECT auth_id FROM users WHERE users.id = reading_history.user_id));

CREATE POLICY "Users manage own bookmarks" ON bookmarks FOR ALL
  USING (auth.uid() = (SELECT auth_id FROM users WHERE users.id = bookmarks.user_id));

-- 3. PUBLIC CATALOG TABLES: Read-only access for authenticated and anonymous clients
ALTER TABLE comics ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Public read-only comics" ON comics FOR SELECT USING (true);
```

5.4 On-Device 4-Tier Storage & Bidirectional Sync Strategy

Table 11 defines the **4-Tier Mobile Storage Strategy** deployed on the client device. Crucially, **binary image and audio files are never stored as SQLite BLOBs** (which would bloat database pages and cause OOM crashes, violating NFR-03 and NFR-06); all media assets reside on the sandboxed file system while SQLite stores only indexed relative file paths.

Table 11: 4-Tier On-Device Mobile Persistence and Eviction Strategy.

Storage Tier	Engine / API	Persisted Data	Security & Eviction Policy
Tier 1: Secure Credentials	`flutter_secure_storage`	Supabase JWT `access_token`, `refresh_token`	**AES-256 hardware encryption** (Keystore / Keychain); wiped immediately on *Sign Out* (S08).
Tier 2: Relational Metadata	`Drift` (SQLite ORM)	Cached metadata, `reading_history`, `bookmarks`, `offline_downloads`, sync queue	App-private sandboxed DB; retained across restarts; indexed for $\leq$ 50 ms queries (NFR-09).
Tier 3: Offline Downloads	`Application DocumentsDir`	Downloaded chapter pages (`.webp`) for offline reading	Protected from automatic cache eviction; user-managed via S07 (*Offline Reading*).
Tier 4: Dynamic Disk Cache	`Temporary Directory` + `cached_network_image`	Transient online covers, preloaded chapter pages, audio buffers	**LRU Eviction (NFR-07):** hard cap at 500 MB (auto-pruned to 350 MB) or manual clear on S08.

Bidirectional Offline-First Synchronisation Algorithm: Whenever the user reads or bookmarks while offline, records are written to local SQLite with `is_synced = FALSE` and `last_read_at = NOW()`. Upon network restoration (detected via `ConnectivityPlus`), a background sync worker batches all unsynced rows to Supabase via `POST /rest/v1/reading_history` using a **Last-Write-Wins (LWW)** timestamp resolution rule before setting `is_synced = TRUE`.

5.5 RESTful API Contract Specification & Error Code Dictionary

All remote communications use `HTTPS / TLS 1.3` with JSON payloads (`application/json; charset=utf-8`). Table 12 summarises the seven core API endpoints across **MangaDex REST API v5** (`https://api.mangadex.org`) and **Supabase Backend Services** (`https://[project-id].supabase.co`), followed by representative payload schemas and the standardized Error Code Dictionary (Table 13).

Table 12: Catalogue of the 7 RESTful API Endpoints across MangaDex v5 and Supabase Backend.

API ID	Method & Endpoint Path	Provider	Request Parameters / Headers & Purpose	FR / NFR
API 01	`GET /manga`	MangaDex v5	Query: `title`, `includedTags[]`, `limit=20`, `offset`, `order[rating]=desc`. Searches & filters comics.	FR-01, FR-08, FR-11
API 02	`GET /manga/{id}/feed`	MangaDex v5	Query: `translatedLanguage[]=en` `order[chapter]=asc`, `limit=100`. Fetches ordered chapter feed.	FR-02, FR-10
API 03	`GET /at-home/ server/{chapterId}`	MangaDex@Home	Resolves CDN `baseUrl`, `hash`, high-res `data[]`, and compressed `dataSaver[]` page filenames.	FR-02, FR-06, NFR-18
API 04	`POST /auth/v1/token` `POST /auth/v1/signup`	Supabase Auth	Header: `apikey`. Body: `{email, password}` or `{refresh_token}`. Issues JWT Bearer session.	FR-00, NFR-05, NFR-10
API 05	`POST /rest/v1/ reading_history`	Supabase REST	Header: `Bearer JWT`, `Prefer: resolution=merge-duplicates`. Upserts page-level reading progress.	FR-04, NFR-09
API 06	`GET | POST | DELETE /rest/v1/bookmarks`	Supabase REST	Header: `Bearer JWT`. CRUD operations on user's cloud-synced *Favorites* library.	FR-07, FR-10
API 07	`GET /rest/v1/ ambient_tracks`	Supabase REST	Query: `category=eq.{genre}`. Returns genre-matched instrumental MP3 streaming URLs.	FR-05, FR-09

Listing 5: Representative Request/Response Schemas for API 03 (MangaDex@Home CDN), API 04 (Supabase Auth), API 05 (Reading Progress Upsert), and API 07 (Genre Ambient Audio).

```
// 1. API 03 Response: GET https://api.mangadex.org/at-home/server/{
    chapterId} (HTTP 200 OK)
// Page URL construction: {baseUrl}/data/{hash}/{fileName} (or /data-saver
    / on 4G cellular)
{
  "result": "ok",
  "baseUrl": "https://cmdxd98sb0x3yprd.mangadex.network",
```

```
  "chapter": {
    "hash": "a823f99c15e8bc42f2b3b0d23c10",
    "data": ["page_001.webp", "page_002.webp", "page_042.webp"],
    "dataSaver": ["page_001.jpg", "page_002.jpg", "page_042.jpg"]
  }
}

// 2. API 04 Response: POST https://[project].supabase.co/auth/v1/token?grant_type=password (HTTP 200 OK)
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 3600,
  "refresh_token": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "user": {
    "id": "e4eaaaf2-d142-11e1-b3e4-080027620cdd",
    "email": "reader.kiet@gmail.com",
    "user_metadata": { "display_name": "Kiet Nguyen" }
  }
}

// 3. API 05 Request & Response: POST /rest/v1/reading_history (HTTP 201 Created)
{
  "user_id": "e4eaaaf2-d142-11e1-b3e4-080027620cdd",
  "chapter_id": "chapter-uuid-024",
  "last_page_read": 18,
  "read_percentage": 42.8,
  "is_completed": false,
  "last_read_at": "2026-11-04T12:00:00Z"
}

// 4. API 07 Response: GET /rest/v1/ambient_tracks?category=eq.fantasy (HTTP 200 OK)
[
  {
    "id": "track-uuid-fantasy-01",
    "title": "Winds of Aureole (Instrumental)",
    "category": "fantasy",
    "audio_url": "https://[project].supabase.co/storage/v1/object/public/ambient-audio/winds_aureole.mp3",
    "duration_seconds": 192,
    "loop_enabled": true
  }
]
```

Table 13: System Error Code Dictionary and Deterministic UI/UX Recovery Actions.

HTTP Status	Internal Error Code	Technical Meaning	Presentation Layer (BLoC / UI) Recovery Action
200 / 201	`SUCCESS`	Request succeeded	Emit `LoadedState`; render data and update local SQLite cache.
204	`NO_CONTENT`	Resource deleted cleanly	Remove bookmark/item from UI state immediately.
400	`ERR_BAD_REQUEST`	Malformed input or invalid credentials	Display inline field validation error on `S01` while preserving user input.
401	`ERR_UNAUTHORIZED`	Expired or missing JWT token	Trigger silent token refresh (`API 04`); if refresh fails, route to `S01 Sign In`.
403	`ERR_FORBIDDEN`	Supabase RLS policy violation	Block unauthorized cross-user mutation and log security event.
404	`ERR_NOT_FOUND`	Comic or chapter unavailable	Render clean Empty State with a back navigation button.
429	`ERR_RATE_LIMITED`	Exceeded MangaDex 5 req/s limit	`Dio` interceptor executes automatic exponential backoff respecting `Retry-After`.
500	`ERR_INTERNAL_SERVER`	Upstream server fault	Display non-blocking SnackBar with a 1-tap *Retry* action.
0 (No Net)	`ERR_NETWORK_DISCONNECTED`	Complete loss of Internet connectivity	**Automatic Offline Fallback (`NFR-08` $\leq$ 300 ms):** serve cached chapters from SQLite or display `S09` routing to `S07 Offline Library`.

6 UI/UX Design & High-Fidelity Mockups

6.1 Core Design Philosophy: "Less is More, Content-First"

The user interface of **Naomi** is governed by a strict **Content-First Minimalist Philosophy** combining **Material Design 3** and **Apple Human Interface Guidelines (HIG)** across six non-negotiable design rules:

1. **Elimination of Visual Clutter:** Zero marketing slogans, zero mood-tag bloat, and zero promotional banners; every screen displays only the exact controls and artwork required for the task.
2. **Icon-Only Reader Toolbar:** All action controls on the `S05` Reader top bar use intuitive iconography without text labels to maximise unobstructed manga artwork area (100% viewport canvas).
3. **Muted Minimal Color Palette:** Avoids neon gradients or multi-hue noise; uses calm monochrome neutrals paired with a single soft smoky-lavender accent (`#7066A6`) so colorful comic covers and black-and-white manga ink take 100% visual focus.
4. **100% International English UI Copy:** All labels, buttons, status indicators, and empty/error states use clean English microcopy (*Sign In, Continue, Favorites, Offline Reading, Oops! No internet connection*).
5. **100% Authentic Manga Artwork:** All 12 frames render authentic MangaDex cover and page artwork (*Frieren: Beyond Journey's End, Delicious in Dungeon, Yokohama Kaidashi Kikou, Witch Hat Atelier, The Apothecary Diaries, Mushishi*), never grey wireframe boxes.
6. **Full-Chapter Pre-Buffering (`S10`):** Preloads 100% of chapter pages prior to opening `S05` to guarantee zero mid-chapter loading spinners.

Interactive Prototype & Figma Artifacts:

- **Figma Design File (390 × 844 px viewport):** `https://www.figma.com/design/flQOgQxYD3xWOUEpsnAUlM?node-id=1-3`
- **Local Interactive HTML5/JS Prototype:** Included in the repository at `document/naomi-ui-prototype/index.html` (supporting live frame switching across all 12 screens).

6.2 Minimal Design Tokens & Accessibility Standards

Table 14 defines the complete colour, typography, and spatial token system of **Naomi**, engineered to exceed **WCAG 2.1 AA/AAA** contrast ratios (13.5:1 on light surfaces; 14.2:1 on dark reader surfaces) and $\geq 48 \times 48$ dp touch target ergonomics (`NFR-15`).

Table 14: Minimalist Muted Soft Palette, Spatial Grid, and Accessibility Tokens.

Token Name	Hex / Value	Swatch	Semantic Usage & Accessibility Role
`bg/canvas`	`#F8F8FC`		Soft off-white primary background preventing eye glare during daytime browsing (`S01-S04, S07-S09, S11`).
`bg/surface` & `subtle`	`#FFFFFF` / `#EFEFF4`		Elevated card containers, input fields, and subtle dividers.
`text/primary` & `secondary`	`#1C1C26` / `#757588`		Deep charcoal primary typography (13.5:1 contrast) and muted slate metadata text.
`brand/accent` & `tint`	`#7066A6` / `#F0EFF8`		**Single Soft Accent:** Smoky pastel lavender for primary CTAs, progress bars, and the `S02` Hero card tint.
`reader/bg` & `surface`	`#111116` / `rgba(22,22,28,.9)`		True AMOLED dark reader backdrop (`S05, S06, S10`) saving up to 30% OLED power (`NFR-16`).
`semantic/fav, ok, off`	`#E05353` `#34A853` `#D97706`		Muted coral red (*Favorite*), sage green (*Saved / Download 68%*), and soft amber (*Offline Warning*).
Grid, Radii & Touch	`8px` grid, `48dp` target	—	Card radius `12px`; Library big buttons `20px`; Dialog/Toolbar `16px`; all icons inside $\geq 48 \times 48$ dp hit boxes.

6.3 Complete Screen Navigation & User Flow Architecture

Figure 10 maps the complete end-to-end navigation graph across all 12 screens (`S01` through `S11`, including `S03b`), covering normal reading flows, audio sheet overlays, chapter preloading, active download progress, and offline network loss recovery.

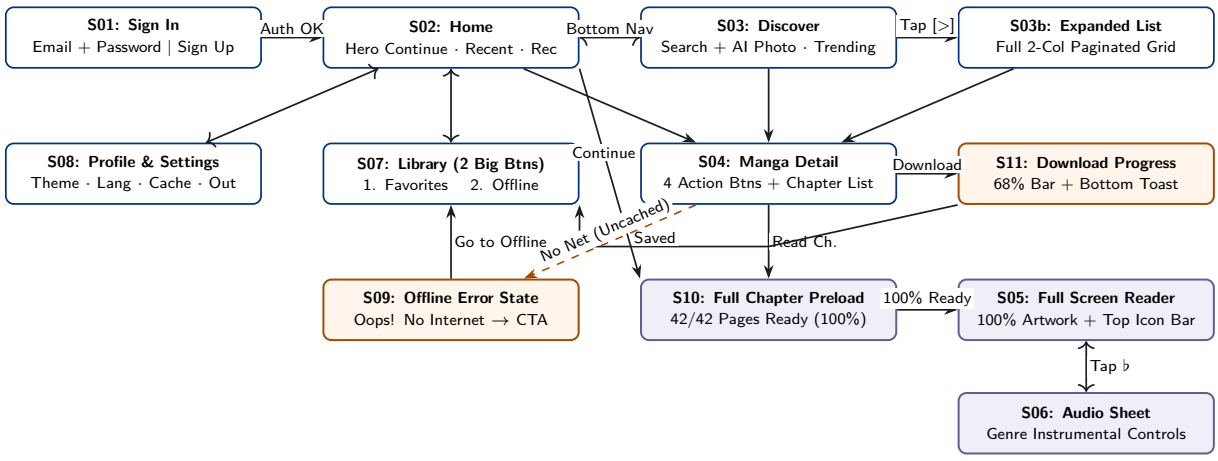
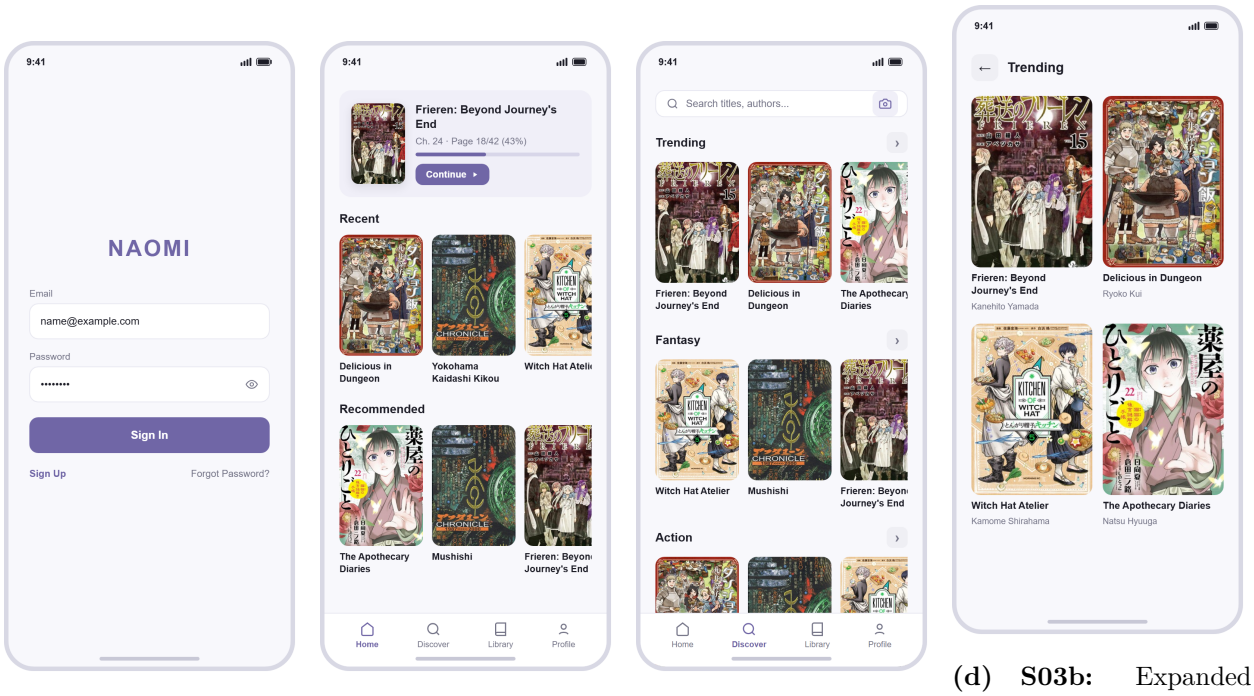


Figure 10: Complete User Flow and Screen Navigation Graph across all 12 frames (`S01` to `S11`).

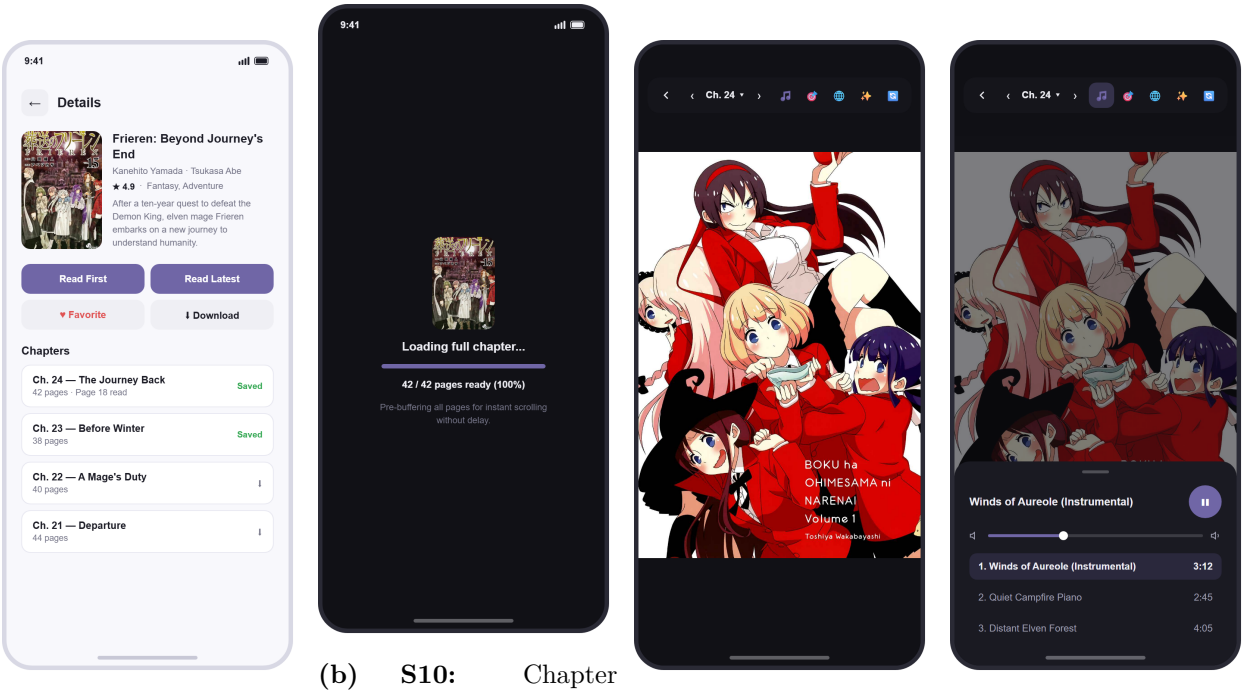
6.4 High-Fidelity Mobile Mockups & Screen-by-Screen Specifications

Figures 11, 12, and 13 present the 12 high-fidelity 390 × 844 px mobile mockups rendered directly from the **Naomi** design system, followed by Table 15 detailing the functional components and state coverage of every frame.



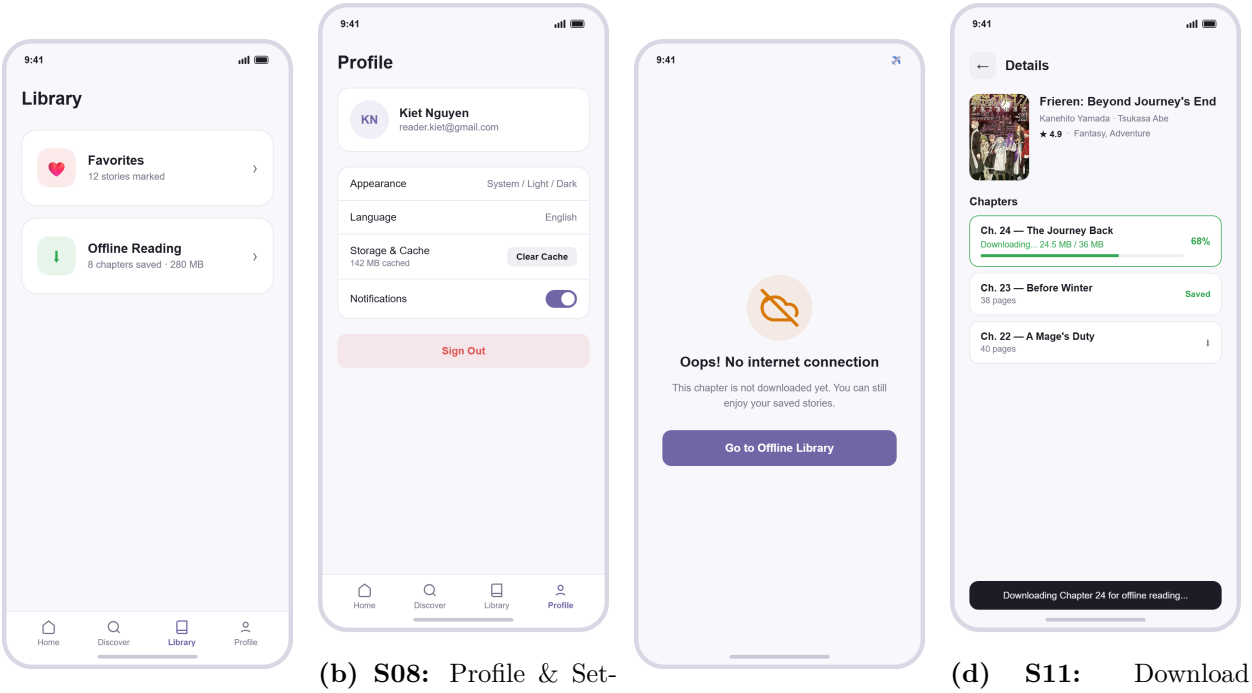
(a) S01: Sign In **(b) S02:** Home Feed **(c) S03:** Discover **(d) S03b:** Expanded Grid

Figure 11: High-Fidelity Mockups (Part 1/3): Authentication (`S01`), 3-Block Home Feed (`S02`), Discover with AI Photo Search (`S03`), and Expanded Category Grid (`S03b`).



(a) **S04:** Manga Detail (b) **S10:** Chapter Preload (c) **S05:** Full Reader (d) **S06:** Audio Sheet

Figure 12: High-Fidelity Mockups (Part 2/3): Manga Detail (S04), Full-Chapter Preload State (S10), 100% Full-Screen Reader (S05), and Genre Instrumental Audio Sheet (S06).



(a) **S07:** Library Hub (b) **S08:** Profile & Settings (c) **S09:** Offline Error (d) **S11:** Download (68%)

Figure 13: High-Fidelity Mockups (Part 3/3): Two-Button Library (S07), Profile & Settings (S08), Offline Network Error State (S09), and Real-Time Download Progress State (S11).

Table 15: Comprehensive Specification of all 12 UI/UX Frames (S01 to S11) including Primary, Loading, Error, and Offline States.

Frame	Screen Title	UI State Role	Key Components, Interactions & Design Rules
S01	**Sign In**	Auth Entry	Bold lavender **NAOMI** wordmark (#7066A6), Email & Password inputs (12px radius), primary **Sign In** button, **Sign Up** and **Forgot Password?** links. Zero third-party social clutter.
S02	**Home**	Primary Feed	Strictly 3 blocks: (1) Lavender-tinted **Continue Reading** Hero card (*Frieren Ch. 24 · Page 18/42 (43%)*) with 1-tap **Continue** ▶ pill; (2) **Recent** horizontal scroll; (3) **Recommended** horizontal scroll + 4-tab Bottom Nav.
S03	**Discover**	Search & Browse	Search input (*"Search titles, authors..."*) with trailing **Camera Icon** (invokes zero-permission OS Photo Picker for FR-11), followed by **Trending** and **Genre** (*Fantasy, Action*) horizontal rows with [>] expand buttons.
S03b	**Expanded List**	Category Grid	Full-screen 2-column vertical grid triggered by tapping [>] on S03; includes Back button (←) and category header.
S04	**Manga Detail**	Series Hub	Cover artwork, title, author (*Kanehito Yamada · Tsukasa Abe*), rating (⋆ 4.9), genres, synopsis, 4 action buttons (**Read First**, **Read Latest**, ♡ **Favorite**, ↓ **Download**), and full chapter list with **Saved** badges.
S10	**Full Chapter Preload**	**Loading State**	AMOLED dark screen (#111116) with pulsing manga cover, *"Loading full chapter... 42 / 42 pages ready (100%)"* progress bar, pre-buffering all pages before opening S05.
S05	**Full Screen Reader**	Core Reader	100% full-screen manga artwork with a single translucent top bar: Back (<), Chapter switcher (< Ch. 24 ▼ >), and 5 pure icons: Music (♭), Focus Mode (⊙), Language (⊕), AI Translate (⋆), and Mode Toggle (⥃).
S06	**Audio Sheet**	Modal Overlay	Minimalist bottom sheet over S05: active genre instrumental title (*Winds of Aureole*), 44 px Play/Pause button, smooth volume slider, and 3-row selectable track list for the active genre.
S07	**Library**	Collection Hub	Exactly **2 Large** 20 **px-Radius Action Buttons**: (1) ♡ **Favorites** (*12 stories marked*) and (2) ↓ **Offline Reading** (*8 chapters saved · 280 MB*).
S08	**Profile & Settings**	User & System	Avatar (KN), *Kiet Nguyen (reader.kiet@gmail.com)*, **Appearance** (*System / Light / Dark*), **Language** (*English / Tieng Viet*), **Storage & Cache** (*142 MB cached + Clear Cache*), **Notifications**, and **Sign Out**.
S09	**Offline Error State**	**Error / Empty Fallback**	Triggered within $\leq$ 300 ms when tapping an uncached chapter without Internet: amber offline cloud icon, *"Oops! No internet connection"*, and primary CTA button **Go to Offline Library** → S07.
S11	**Download in Progress**	**Active Progress State**	Highlights Chapter 24 row in sage green (#34A853) with live progress bar (*"Downloading... 24.5 MB / 36 MB – 68%"*) and bottom toast banner (*"Downloading Chapter 24 for offline reading..."*).

7 Traceability Matrix & Document Quality Assurance

7.1 End-to-End Bidirectional Traceability Methodology

In accordance with software engineering best practices (*Sommerville, Software Engineering, 10th Ed., Chapter 4 – Requirements Management*) and Rubric Criterion 1.7, a production-grade specification must establish **End-to-End Bidirectional Traceability** across all five engineering layers:

Elicitation (Sec. 1.1) ⟷ FR/NFR (Sec. 1.2, 1.3) ⟷ UC/US (Sec. 1.2)
⟷ UI Mockups (Sec. 1.6) ⟷ Architecture, DB & APIs (Sec. 1.4, 1.5)

This rigorous linkage guarantees two vital properties for Assignment 02:

1. **Zero Gold-Plating (Forward Traceability):** Every UI frame (`S01`–`S11`), BLoC state controller, relational database table, and REST API endpoint exists solely to fulfil one or more empirically elicited user requirements.
2. **100% Acceptance Coverage (Backward Traceability):** Every `MUST-HAVE`, `SHOULD-HAVE`, and `COULD-HAVE` requirement is backed by concrete architectural components, normalised database tables, API contracts, and testable Gherkin BDD scenarios.

7.2 Functional Requirements Master Traceability Matrix

Table 16 maps 100% of the Functional Requirements (`FR-00` through `FR-16`) across their elicitation origins, Use Cases/User Stories, UI Mockup Screens, In-App Clean Architecture components, Database Tables, and REST API endpoints.

Table 16: Master Bidirectional Traceability Matrix for Functional Requirements (`FR-00` to `FR-16`).

FR ID	Requirement Summary	MoSCoW	Elicitation	UC / US	UI Screen	Clean Architecture	DB & APIs
FR-00	User Account Auth (Sign Up, Sign In, Reset)	**MUST**	Pain Point #7 (Cloud sync)	`UC-00` `US-00`	**S01**, **S08**	`AuthBloc` → `SignInUseCase` → `AuthRepoImpl` → `SecureStorage`	Table `users` **API 04** (`/auth/v1/token`)
FR-01	Discover & Search (Trending, Genres, Expand)	**MUST**	Pain Point #3, Q2 (75% catalog)	`UC-01` `US-01`	**S03**, **S03b**	`DiscoverBloc` → `SearchComicsUseCase` → `ComicRepoImpl`	`comics`, `genres`, `comic_genres` **API 01** (`GET /manga`)
FR-02	Full-Screen Reader & Focus Mode	**MUST**	Persona 1, Q2 (100% hate ad clutter)	`UC-03`, `UC-05` `US-03`, `US-05`	**S10**, **S05**	`ReaderBloc` → `GetChapterContentUseCase` → `ComicRepoImpl`	`chapters`, `pages` **API 03** (`/at-home/server`)

Continued on next page

Table 16 – continued from previous page

FR ID	Requirement Summary	MoSCoW	Elicitation	UC / US	UI Screen	Clean Architecture	DB & APIs
FR-03	Dual Reader Mode Toggle (Webtoon ↔ Manga)	**MUST**	Persona 1, Q5 (25% mode toggle)	UC-03 US-03	**S05** (Icon ⇆)	ReaderBloc (ToggleModeEvent) → ComicRepoImpl	pages, users.preferences Local Drift SQLite
FR-04	Auto-Save Page Progress & Instant Resume	**MUST**	Persona 3, Q5 (75% page history)	UC-05 US-05	**S02** (Hero), **S05**	ReaderBloc → Save ReadingProgressUseCase → DriftDB	reading_history **API 05** (/reading_history)
FR-05	Genre-Matched Instrumental Ambient Audio	**MUST**	Persona 1, Q3 (100%), Q4 (3.75/5)	UC-04 US-04	**S05** (Icon **S06**	AudioCubit → GetGenre AudioUseCase → JustAudioService	ambient_tracks, genres **API 07** (/ambient_tracks)
FR-06	Offline Chapter Download with Live Progress	**MUST**	Persona 2, Q2 (50% offline/VIP)	UC-06 US-06	**S04**, **S11**	DownloadBloc → DownloadMgrService → LocalFileStorage	offline_downloads **API 03** (/at-home/server)
FR-07	Personal Library (Favorites & Offline Reading)	**MUST**	Persona 2, Q5 (Library categories)	UC-07 US-07	**S07** (2 Big Btns)	LibraryBloc → Get LibraryUseCase → ComicRepoImpl	bookmarks, offline_downloads **API 06** (/bookmarks)
FR-08	Consolidated Home Feed (Continue, Recent, Rec)	**SHOULD**	Persona 3, Q5 (75% genre recs)	UC-01, UC-05 US-01, US-05	**S02**	HomeBloc → GetHome FeedUseCase → ComicRepoImpl	reading_history, comics, genres **API 01 & API 05**
FR-09	Minimalist Audio Sheet Controls & Focus Pause	**SHOULD**	Persona 1, Q4 (Ambient controls)	UC-04 US-04	**S06**	AudioCubit → JustAudioService (AudioSession)	ambient_tracks **API 07** (/ambient_tracks)
FR-10	Comic Detail View & 4 Quick Chapter Actions	**SHOULD**	Persona 2, Persona 3	UC-02 US-02	**S04**	ComicDetailBloc → Get ComicDetailUseCase → ComicRepoImpl	comics, chapters, bookmarks **API 02 & API 06**
FR-11	Zero-Permission Visual Search (Photo Picker)	**COULD**	Persona 2, Q5 (50% visual search)	UC-01 US-01	**S03** (Camera)	DiscoverBloc → Visual SearchUseCase → SystemPhotoPicker	comics **API 01** (GET /manga)
FR-12	In-Place AI Chapter Translation	**COULD**	Persona 3, Q5 (100% AI translate)	UC-03 US-03	**S05** (Icon ⋆)	ReaderBloc (TranslateEvent) → AiTranslationService	pages (In-memory translated overlay cache)
FR-13	Offline Network Loss Error State & Routing	**SHOULD**	Persona 2, Q2 (50% network drop)	UC-06 US-06	**S09** → **S07**	ConnectivityCubit (ConnectivityPlus) → ReaderBloc	offline_downloads (Local SQLite Drift)
FR-14	Profile & System Settings (Theme, Lang, Cache)	**SHOULD**	Q1 (Day/Night), Pain Point #4	UC-08 US-08	**S08**	SettingsCubit → CacheMgrService & AuthBloc	users.preferences (JSONB) + Tier 4 LRU Cache
FR-15	In-App Purchases (IAP) / Virtual Coins	**WON'T**	100% Free, Non-Commercial Rule	Excluded	None	Excluded (Zero payment/ad SDKs)	None
FR-16	Social Feed & 1-on-1 Direct Messaging	**WON'T**	Content-First Focus Philosophy	Excluded	None	Excluded	None

7.3 Non-Functional Requirements Traceability Matrix

Table 17 traces all 18 Non-Functional Requirements (NFR-01 to NFR-18) to their corresponding UI screens, architectural/database mechanisms, and Assignment 02 QA verification tools.

Table 17: Bidirectional Traceability Matrix for Non-Functional Requirements (NFR-01 to NFR-18).

NFR ID	SMART Criterion & Threshold	UI Screens	Architectural & Data Engineering Mechanism (Sec. 4 & 5)	Verification Tool
NFR-01	Cold Start Time $\leq$ 2.0 s	**S01, S02**	Lazy service DI; render initial S02 frame from local SQLite Drift prior to network sync.	Android Vitals / DevTools
NFR-02	Warm Start Time $\leq$ 0.8 s	All Screens	Retain BLoC state in memory; fast AES-256 session restore via FlutterSecureStorage.	Flutter DevTools
NFR-03	Reader Scrolling $\geq$ 60 FPS ($\leq$ 16.6 ms)	**S05**	Impeller/Skia engine + virtualised viewport list + file-system page storage (no SQLite BLOBs).	Performance Overlay
NFR-04	Chapter Preload Initiation $\leq$ 1.5 s	**S10 $\rightarrow$ S05**	Concurrent page pre-buffering from MangaDex@Home CDN (API 03) on S10.	Network Profiler
NFR-05	Auth Response Time $\leq$ 1.0 s	**S01**	Lightweight POST /auth/v1/token (API 04) endpoint over TLS 1.3.	Postman / Profiler
NFR-06	Peak Runtime RAM $\leq$ 250 MB	**S05, S10**	Recycle off-screen GPU page textures outside the $\pm$2-page viewport window.	Android Memory Profiler
NFR-07	Temporary Disk Cache $\leq$ 500 MB	**S08**	Tier 4 LRU cache eviction worker auto-pruning to 350 MB + manual *Clear Cache* on S08.	File Storage Inspector
NFR-08	Offline Fallback Latency $\leq$ 300 ms	**S09 $\rightarrow$ S07**	Reactive ConnectivityPlus stream listener routing uncached offline requests to S09.	Connectivity Listener
NFR-09	Local SQLite I/O $\leq$ 50 ms/query	**S02, S05, S07**	Indexed foreign/unique keys (user_id, chapter_id) and asynchronous Drift isolates.	SQLite Benchmark
NFR-10	Hardware Token Encryption **AES-256**	**S01, S08**	Tier 1 FlutterSecureStorage backed by Android Keystore and iOS Keychain.	Security / Root Audit
NFR-11	Transport Security **100% TLS 1.3**	**API 01–07**	usesCleartextTraffic="false" in AndroidManifest.xml + Supabase PostgreSQL RLS.	Charles Proxy / Audit
NFR-12	**0 Dangerous OS Permissions**	**S03**	OS System Photo Picker for Visual Search (FR-11); zero Camera/Storage/GPS permissions.	AndroidManifest.xml
NFR-13	Graceful Permission Denial Handling	**S08, S11**	Non-crashing runtime handling of POST_NOTIFICATIONS; fallback to in-app toast on S11.	Manual UI Test Case
NFR-14	Android 8.0+ (API 26–34) & iOS 13+	All Screens	Configured in build.gradle (minSdkVersion 26, targetSdkVersion 34) & Podfile.	Firebase Test Lab
NFR-15	Responsive 16:9–21:9 & SafeArea	**S01–S11**	Global SafeArea wrapper, 8 px grid, and $\geq 48 \times 48$ dp touch targets.	Layout Inspector
NFR-16	Battery Drain $\leq$ 8% / hour	**S05, S06, S08**	AMOLED dark reader background (#111116) + hardware audio decoding via just_audio.	Battery Historian
NFR-17	Audio Focus Pause $\leq$ 100 ms	**S05, S06**	AudioSession interruption listener pausing playback on phone call or backgrounding.	Audio Focus Test
NFR-18	4G Data Saver 40%–60% Reduction	**S05, S10, S11**	Automatic selection of dataSaver[] compressed arrays from API 03 on cellular networks.	Network Traffic Monitor

7.4 Document Quality & Consistency Assurance Checklist

Table 18 summarises the internal verification audit conducted across all seven sections of this consolidated specification document.

Table 18: Cross-Section Consistency and Quality Assurance Verification Checklist.

Quality Verification Dimension	Audit Status	Concrete Evidence of Cross-Section Synchronisation
1. Empirical Survey ↔ Personas ↔ MoSCoW	**Verified 100%**	All statistics in Section 1 match the 4 participant forms in `khao_sat_4_nguoi.pdf`. Persona 1 (*Kiet Nguyen – reader.kiet@gmail.com*) is used consistently across Section 1, `API 04` (Section 5), and Screen `S08` (Section 6).
2. Identifier Synchronisation (`FR-00..16`, `UC-00..08`, `US-00..08`)	**Verified 100%**	All 17 Functional Requirements and 9 Use Cases align 1-to-1 across the MoSCoW matrix, UML Use Case Diagram (Figure 2), Gherkin BDD scenarios, and Master Traceability Matrix (Table 16).
3. SMART NFR Thresholds (`NFR-01..18`)	**Verified 100%**	All 18 NFR metrics (60 FPS, $\leq$ 250 MB RAM, $\leq$ 500 MB LRU cache, $\leq$ 300 ms offline fallback, $\leq$ 50 ms SQLite, AES-256, 0 dangerous permissions) are consistently enforced across Sections 3, 4, 5, and 7.
4. Chen Conceptual ERD ↔ 3NF/BCNF Physical Schema	**Verified 100%**	Both ERD diagrams (Figures 8–9) and SQL specifications model `pages` as a weak entity with composite PK `(chapter_id, page_number)` and eliminate `comic_id` from `reading_history` and `offline_downloads` to satisfy strict 3NF/BCNF.
5. UI/UX Specification ↔ 12 High-Fidelity Mockups	**Verified 100%**	All 12 frames (`S01`–`S11`, including `S03b`) are rendered at 390 × 844 px with real MangaDex artwork, English copy, Muted Minimal tokens (`#7066A6`), and complete secondary states (`S10` Preload, `S09` Offline Error, `S11` Download Progress).